

大乘软件 V3.0 源程序代码

=====

```
// =====
// 文件: lib/core/config/app_config_template.dart
// =====

/// 应用配置
/// 统一管理所有配置项
class AppConfig {
  // 私有构造函数
  AppConfig._();

  // 单例
  static final AppConfig instance = AppConfig._();

  // 环境配置
  static const String environment = String.fromEnvironment('ENV', defaultValue: 'prod...

  // 是否为生产环境
  static bool get isProduction => environment == 'production';
  static bool get isDevelopment => environment == 'development';
  static bool get isStaging => environment == 'staging';

  // API配置
  static const String productionApiUrl = 'https://ombhrum.com';
  static const String stagingApiUrl = 'https://staging.ombhrum.com';
  static const String developmentApiUrl = 'http://localhost:8787';

  // 根据环境获取API URL
  static String get apiUrl {
    switch (environment) {
      case 'production':
        return productionApiUrl;
      case 'staging':
        return stagingApiUrl;
      case 'development':
        return developmentApiUrl;
      default:
        return productionApiUrl;
    }
  }

  // API端点
  static const String authEndpoint = '/api/auth';
  static const String membershipEndpoint = '/api/membership';
  static const String transferEndpoint = '/api/transfer';
  static const String leaderboardEndpoint = '/api/leaderboard';

  // 应用信息
  static const String appName = '全球法布施';
  static const String appVersion = '1.0.0';
  static const int appBuildNumber = 1;

  // 功能开关
  static const bool enableFirebaseAuth = true;
  static const bool enableAlipay = true;
  static const bool enableVideoFeed = true;
  static const bool enableDebugMode = !isProduction;

  // 传输配置
  static const int fileChunkSize = 1024;
  static const int maxRetryCount = 3;
  static const int timeoutDuration = 5000;
  static const int maxConcurrentTransfers = 5;
```

```
// 缓存配置
static const int cacheMaxAge = 7; // 天
static const int maxCacheSize = 100; // MB

// 分页配置
static const int defaultPageSize = 20;
static const int maxPageSize = 100;

// 支持的国家代码
static const List<String> supportedCountries = [
    'ALL',
    'US',
    'CN',
    'IN',
    'ID',
    'BR',
    'PK',
    'NG',
    'BD',
    'RU',
    'MX',
    'JP',
    'ET',
    'PH',
    'EG',
    'VN',
    'CD',
    'TR',
    'IR',
    'DE',
    'TH',
    'GB',
    'FR',
    'IT',
    'ZA',
    'TZ',
    'MM',
    'KE',
    'KR',
    'CO',
    'ES',
    'UG',
    'AR',
    'DZ',
    'SD',
    'UA',
    'CA',
    'PL',
    'MA',
    'SA',
    'UZ',
    'PE',
    'AF',
    'MY',
    'AO',
    'MZ',
    'GH',
    'YE',
    'NP',
    'VE',
    'MG',
    'AU',
    'KP',
    'CM',
    'NE',
```

'TW',
'LK',
'BF',
'ML',
'RO',
'MW',
'CL',
'KZ',
'ZM',
'GT',
'EC',
'SY',
'NL',
'SN',
'KH',
'TD',
'SO',
'ZW',
'GN',
'RW',
'BJ',
'TN',
'BI',
'BO',
'HT',
'BE',
'CU',
'SS',
'DO',
'CZ',
'GR',
'JO',
'PT',
'AZ',
'SE',
'HN',
'HU',
'TJ',
'AE',
'BY',
'IL',
'TG',
'AT',
'RS',
'PG',
'CH',
'SL',
'HK',
'LA',
'LY',
'BG',
'KG',
'NI',
'ER',
'TM',
'SG',
'DK',
'FI',
'CG',
'SK',
'NO',
'OM',
'PS',
'CR',
'LB',
'NZ',

'CF',
'IE',
'LR',
'MR',
'PA',
'GE',
'UY',
'BA',
'MN',
'AM',
'JM',
'QA',
'AL',
'PR',
'LT',
'NA',
'GM',
'BW',
'GA',
'SI',
'GW',
'HR',
'MK',
'LS',
'KW',
'XK',
'TT',
'EE',
'MU',
'SZ',
'DJ',
'FJ',
'RE',
'CY',
'BH',
'KM',
'GQ',
'BT',
'ME',
'SB',
'MO',
'LU',
'SR',
'CV',
'MV',
'MT',
'BN',
'BZ',
'IS',
'BB',
'BS',
'PF',
'VU',
'NC',
'GY',
'ST',
'WS',
'LC',
'GD',
'TO',
'VC',
'KI',
'MH',
'FM',
'PW',
'NR',

```
'TV',
];

// 日志配置
static const bool enableLogging = true;
static const bool enableNetworkLogging = !isProduction;
static const bool enablePerformanceLogging = !isProduction;
}

// =====
// 文件: lib/core/config/app_config.dart
// =====

import 'package:flutter/foundation.dart';

/// 应用配置 - 统一管理所有配置项
class AppConfig {
  AppConfig._();

  static final AppConfig instance = AppConfig._();

  // 环境配置
  static const String environment = String.fromEnvironment('ENV', defaultValue: 'prod...

  static bool get isProduction {
    if (kIsWeb) {
      final currentUrl = Uri.base.toString();
      if (currentUrl.contains('fabushi-flutter-web-dev') || currentUrl.contains('loca...
        return false;
      }
      if (currentUrl.contains('fabushi-flutter-web-prod')) {
        return true;
      }
    }
    return environment == 'production';
  }

  static bool get isDevelopment => !isProduction;
  static bool get isStaging => environment == 'staging';
  static bool get isWeb => kIsWeb;

  // API配置
  static const String primaryBackendUrl = 'https://flutter.ombhrum.com';
  static const String cloudflareWorkerProdUrl = 'https://flutter.ombhrum.com';
  static const String cloudflareWorkerDevUrl = 'https://flutter.ombhrum.com';
  static const String localDevUrl = 'http://localhost:8787';

  static String get currentBackendUrl {
    // 统一使用 flutter.ombhrum.com 作为后端地址
    return primaryBackendUrl;
  }

  static String get apiUrl => currentBackendUrl;

  // 应用信息
  static const String appName = '全球法布施';
  static const String appVersion = '1.0.0';
  static const int appBuildNumber = 1;

  // 功能开关
  static const bool enableFirebaseAuth = true;
  static const bool enableAlipay = true;
  static const bool enableVideoFeed = true;
  static bool get enableDebugMode => !isProduction;
```

```
// 传输配置
static const int fileChunkSize = 1024;
static const int maxRetryCount = 3;
static const int timeoutDuration = 5000;
static const int maxConcurrentTransfers = 5;

// 缓存配置
static const int cacheMaxAge = 7;
static const int maxCacheSize = 100;

// 分页配置
static const int defaultPageSize = 20;
static const int maxPageSize = 100;

// 超时配置
static final Duration requestTimeout = const Duration(seconds: 30);
static final Duration connectTimeout = const Duration(seconds: 10);

// 重试配置
static const int maxRetries = 3;
static final Duration retryDelay = const Duration(seconds: 1);

// 日志配置
static const bool enableLogging = true;
static bool get enableNetworkLogging => !isProduction;
static bool get enablePerformanceLogging => !isProduction;

// 存储键名
static const String tokenStorageKey = 'auth_token';
static const String userInfoStorageKey = 'user_info';
static const String backendUrlStorageKey = 'backend_url';
static const String testModeStorageKey = 'test_mode';

// API端点
static String get loginUrl => '$currentBackendUrl/api/auth/login';
static String get registerUrl => '$currentBackendUrl/api/auth/register';
static String get verifyUrl => '$currentBackendUrl/api/auth/verify';
static String get logoutUrl => '$currentBackendUrl/api/auth/logout';
static String get sendVerificationCodeUrl => '$currentBackendUrl/api/auth/send-veri...
static String get verifyCodeUrl => '$currentBackendUrl/api/auth/verify-code';
static String get forgotPasswordUrl => '$currentBackendUrl/api/auth/forgot-password...
static String get resetPasswordUrl => '$currentBackendUrl/api/auth/reset-password';
static String get userInfoUrl => '$currentBackendUrl/api/auth/user-info';
static String get bindEmailUrl => '$currentBackendUrl/api/auth/bind-email';

static String get alipayCreateOrderUrl => '$currentBackendUrl/api/alipay/create-ord...
static String get alipayQueryOrderUrl => '$currentBackendUrl/api/alipay/query-order...
static String get alipayMembershipStatusUrl => '$currentBackendUrl/api/alipay/check...

static String get stripeMembershipStatusUrl => '$currentBackendUrl/api/stripe/membe...
static String get stripeCreateSubscriptionUrl =>
    '$currentBackendUrl/api/stripe/create-subscription';
static String get stripeSessionStatusUrl => '$currentBackendUrl/api/stripe/session-...

static String get adminCheckStatusUrl => '$currentBackendUrl/api/admin/check-status...
static String get adminCreateRedeemCodeUrl => '$currentBackendUrl/api/admin/create-...
static String get adminRedeemCodesUrl => '$currentBackendUrl/api/admin/redeem-codes...
static String get adminUseRedeemCodeUrl => '$currentBackendUrl/api/admin/use-redeem...
static String get adminPurchaseHistoryUrl => '$currentBackendUrl/api/admin/purchase...
static String get adminRedeemHistoryUrl => '$currentBackendUrl/api/admin/redeem-his...

static String get leaderboardUrl => '$currentBackendUrl/api/leaderboard';
static String get updateTransferDataUrl => '$currentBackendUrl/api/leaderboard/upda...

static String get healthCheckUrl => '$currentBackendUrl/health';
```

```

// iOS 后台保活静音音频
static String get silenceAudioUrl => '$currentBackendUrl/static/audio/silence.mp3';

// 请求头
static Map<String, String> get defaultHeaders => {
  'Content-Type': 'application/json',
  'Accept': 'application/json',
  'User-Agent': 'FabushiApp/${isWeb ? "Web" : "Mobile"}',
};

// 错误消息
static const Map<String, String> errorMessages = {
  'network_error': '网络连接失败，请检查网络设置',
  'server_error': '服务器错误，请稍后重试',
  'timeout_error': '请求超时，请检查网络连接',
  'auth_error': '认证失败，请重新登录',
  'permission_error': '权限不足',
  'validation_error': '数据验证失败',
  'unknown_error': '未知错误，请联系客服',
};

// 备用地址
static List<String> get fallbackUrls {
  return [primaryBackendUrl];
}

// 调试配置
static const bool enableApiLogging = true;
static const bool debugMode = bool.fromEnvironment('DEBUG', defaultValue: false);

static void printConfigInfo() {
  if (kDebugMode) {
    print('=== 应用配置 ===');
    print('环境: ${isProduction ? "生产" : "开发"}');
    print('平台: ${isWeb ? "Web" : "Native"}');
    print('API URL: $currentBackendUrl');
    print('=====');
  }
}

static void printCurrentConfig() => printConfigInfo();
}

// =====
// 文件: lib/core/config/localization/app_localizations.dart
// =====

import 'package:flutter/material.dart';

class AppLocalizations {
  static AppLocalizations of(BuildContext context) {
    return Localizations.of<AppLocalizations>(context, AppLocalizations) ?? AppLocalizations...
  }

  String get follow => '关注';
  String get following => '已关注';
}

// =====
// 文件: lib/core/constants/api_constants_template.dart
// =====

/// API常量定义
class ApiConstants {

```

```
ApiConstants._();

// 认证相关
static const String login = '/api/auth/login';
static const String register = '/api/auth/register';
static const String logout = '/api/auth/logout';
static const String refreshToken = '/api/auth/refresh';
static const String sendVerificationCode = '/api/auth/send-verification-code';
static const String verifyEmail = '/api/auth/verify-email';
static const String resetPassword = '/api/auth/reset-password';
static const String changePassword = '/api/auth/change-password';

// 用户相关
static const String userProfile = '/api/user/profile';
static const String updateProfile = '/api/user/update';
static const String deleteAccount = '/api/user/delete';

// 会员相关
static const String membershipStatus = '/api/membership/status';
static const String membershipPlans = '/api/membership/plans';
static const String subscribe = '/api/membership/subscribe';
static const String cancelSubscription = '/api/membership/cancel';
static const String redeemCode = '/api/redeem/use';
static const String generateCode = '/api/redeem/generate';

// 支付相关
static const String createPayment = '/api/payment/create';
static const String verifyPayment = '/api/payment/verify';
static const String paymentHistory = '/api/payment/history';
static const String alipayAuth = '/api/alipay/auth';
static const String alipayCallback = '/api/alipay/callback';

// 传输相关
static const String startTransfer = '/api/transfer/start';
static const String stopTransfer = '/api/transfer/stop';
static const String transferStatus = '/api/transfer/status';
static const String transferHistory = '/api/transfer/history';
static const String transferStats = '/api/transfer/stats';

// 排行榜相关
static const String leaderboard = '/api/leaderboard';
static const String userRank = '/api/leaderboard/user';
static const String topUsers = '/api/leaderboard/top';

// 内容相关
static const String dharmContent = '/api/dharma/content';
static const String searchContent = '/api/dharma/search';
static const String contentDetail = '/api/dharma/detail';
static const String downloadContent = '/api/dharma/download';

// 视频流相关
static const String videoFeed = '/api/video/feed';
static const String videoDetail = '/api/video/detail';
static const String videoLike = '/api/video/like';
static const String videoComment = '/api/video/comment';

// 服务器配置
static const String serverConfig = '/api/config/servers';
static const String countryServers = '/api/config/countries';

// HTTP Headers
static const String headerContentType = 'Content-Type';
static const String headerAuthorization = 'Authorization';
static const String headerAccept = 'Accept';
static const String headerUserAgent = 'User-Agent';
```



```
// Content Types
static const String contentTypeJson = 'application/json';
static const String contentTypeFormData = 'multipart/form-data';
static const String contentTypeUrlEncoded = 'application/x-www-form-urlencoded';

// 超时配置
static const Duration connectTimeout = Duration(seconds: 30);
static const Duration receiveTimeout = Duration(seconds: 30);
static const Duration sendTimeout = Duration(seconds: 30);

// 重试配置
static const int maxRetries = 3;
static const Duration retryDelay = Duration(seconds: 2);
}

// =====
// 文件: lib/core/constants/api_constants.dart
// =====

/// API常量定义
class ApiConstants {
  ApiConstants._();

  // 认证相关
  static const String login = '/api/auth/login';
  static const String register = '/api/auth/register';
  static const String logout = '/api/auth/logout';
  static const String sendVerificationCode = '/api/auth/send-verification-code';
  static const String verifyEmail = '/api/auth/verify';
  static const String forgotPassword = '/api/auth/forgot-password';
  static const String resetPassword = '/api/auth/reset-password';
  static const String userInfo = '/api/auth/user-info';

  // 会员相关
  static const String membershipStatus = '/api/membership/status';
  static const String redeemCode = '/api/admin/use-redeem-code';

  // 支付相关
  static const String alipayCreateOrder = '/api/alipay/create-order';
  static const String stripeCreateSubscription = '/api/stripe/create-subscription';

  // 传输相关
  static const String globalSend = '/send-global';
  static const String updateTransferData = '/api/leaderboard/update';

  // 排行榜相关
  static const String leaderboard = '/api/leaderboard';

  // 内容相关
  static const String r2List = '/r2?list=true';

  // HTTP Headers
  static const String headerContentType = 'Content-Type';
  static const String headerAuthorization = 'Authorization';
  static const String headerAccept = 'Accept';

  // Content Types
  static const String contentTypeJson = 'application/json';
}

// =====
// 文件: lib/core/constants/app_constants.dart
// =====
```

```
class AppConstants {
  AppConstants._();

  // 应用信息
  static const String appName = '全球法布施';
  static const String appVersion = '1.0.0';

  // 支持的国家代码
  static const List<String> supportedCountries = [
    'ALL',
    'US',
    'CN',
    'IN',
    'ID',
    'BR',
    'PK',
    'NG',
    'BD',
    'RU',
    'MX',
    'JP',
    'ET',
    'PH',
    'EG',
    'VN',
    'CD',
    'TR',
    'IR',
    'DE',
    'TH',
    'GB',
    'FR',
    'IT',
    'ZA',
    'TZ',
    'MM',
    'KE',
    'KR',
    'CO',
  ];

  // 会员类型
  static const String membershipTrial = 'trial';
  static const String membershipPaid = 'paid';
  static const String membershipExpired = 'expired';

  // 传输状态
  static const String transferPending = 'pending';
  static const String transferInProgress = 'in_progress';
  static const String transferCompleted = 'completed';
  static const String transferFailed = 'failed';
}

// =====
// 文件: lib/core/constants/country_servers.dart
// =====

/// 全球国家服务器和名称映射
///
/// 此文件根据原生Web应用中的`global-country-servers.js`文件移植而来。
/// `GLOBAL_COUNTRY_SERVERS` 包含了每个国家代码对应的服务器URL列表。
/// `COUNTRY_NAMES` 提供了国家代码到中文名称的映射。

const Map<String, List<String>> GLOBAL_COUNTRY_SERVERS = {
  'AF': ['https://httpbin.org/post', 'https://jsonplaceholder.typicode.com/posts'],
```

[illegible]

```
],
  'ID': ['https://httpbin.org/post', 'https://reqres.in/api/users'],
  'IR': ['https://httpbin.org/post', 'https://jsonplaceholder.typicode.com/posts'],
  'IQ': ['https://jsonplaceholder.typicode.com/posts', 'https://reqres.in/api/users'],
  'IE': ['https://reqres.in/api/users', 'https://jsonplaceholder.typicode.com/posts'],
  'IM': ['https://httpbin.org/post', 'https://reqres.in/api/posts']
}
```

[illegible]

[illegible]

```

'TT': ['https://httpbin.org/post', 'https://jsonplaceholder.typicode.com/posts'],
'TN': ['https://reqres.in/api/posts', 'https://httpbin.org/post'],
'TR': ['https://reqres.in/api/users', 'https://jsonplaceholder.typicode.com/posts']...
'TM': ['https://httpbin.org/post', 'https://httpbin.org/post'],
'TC': ['https://jsonplaceholder.typicode.com/posts', 'https://reqres.in/api/users']...
'TV': ['https://jsonplaceholder.typicode.com/posts', 'https://reqres.in/api/users']...
'UG': ['https://httpbin.org/post', 'https://jsonplaceholder.typicode.com/posts'],
'UA': ['https://reqres.in/api/posts', 'https://httpbin.org/post'],
'AE': ['https://reqres.in/api/users', 'https://jsonplaceholder.typicode.com/posts']...
'GB': ['https://httpbin.org/post', 'https://httpbin.org/post', 'https://reqres.in/a...
'UM': ['https://jsonplaceholder.typicode.com/posts', 'https://reqres.in/api/users']...
'US': [
  'https://jsonplaceholder.typicode.com/posts',
  'https://reqres.in/api/users',
  'https://httpbin.org/post',
],
'UY': ['https://httpbin.org/post', 'https://jsonplaceholder.typicode.com/posts'],
'UZ': ['https://reqres.in/api/posts', 'https://httpbin.org/post'],
'VU': ['https://reqres.in/api/users', 'https://jsonplaceholder.typicode.com/posts']...
'VE': ['https://httpbin.org/post', 'https://httpbin.org/post'],
'VN': ['https://jsonplaceholder.typicode.com/posts', 'https://reqres.in/api/users']...
'VG': ['https://jsonplaceholder.typicode.com/posts', 'https://reqres.in/api/users']...
'VI': ['https://httpbin.org/post', 'https://jsonplaceholder.typicode.com/posts'],
'WF': ['https://reqres.in/api/posts', 'https://httpbin.org/post'],
'EH': ['https://reqres.in/api/users', 'https://jsonplaceholder.typicode.com/posts']...
'YE': ['https://httpbin.org/post', 'https://httpbin.org/post'],
'ZM': ['https://jsonplaceholder.typicode.com/posts', 'https://reqres.in/api/users']...
'ZW': ['https://jsonplaceholder.typicode.com/posts', 'https://reqres.in/api/users']...
};

```

```
const Map<String, String> COUNTRY_NAMES = {
```

```

'AF': '阿富汗',
'AX': '奥兰群岛',
'AL': '阿尔巴尼亚',
'DZ': '阿尔及利亚',
'AS': '美属萨摩亚',
'AD': '安道尔',
'AO': '安哥拉',
'AI': '安圭拉',
'AQ': '南极洲',
'AG': '安提瓜和巴布达',
'AR': '阿根廷',
'AM': '亚美尼亚',
'AW': '阿鲁巴',
'AU': '澳大利亚',
'AT': '奥地利',
'AZ': '阿塞拜疆',
'BS': '巴哈马',
'BH': '巴林',
'BD': '孟加拉国',
'BB': '巴巴多斯',
'BY': '白俄罗斯',
'BE': '比利时',
'BZ': '伯利兹',
'BJ': '贝宁',
'BM': '百慕大',
'BT': '不丹',
'BO': '玻利维亚',
'BQ': '博内尔、圣尤斯特歇斯和萨巴',
'BA': '波斯尼亚和黑塞哥维那',
'BW': '博茨瓦纳',
'BV': '布维岛',
'BR': '巴西',
'IO': '英属印度洋领地',
'BN': '文莱达鲁萨兰国',
'BG': '保加利亚',

```

'BF': '布基纳法索',
'BI': '布隆迪',
'CV': '佛得角',
'KH': '柬埔寨',
'CM': '喀麦隆',
'CA': '加拿大',
'KY': '开曼群岛',
'CF': '中非共和国',
'TD': '乍得',
'CL': '智利',
'CN': '中国',
'CX': '圣诞岛',
'CC': '科科斯（基林）群岛',
'CO': '哥伦比亚',
'KM': '科摩罗',
'CD': '刚果民主共和国',
'CG': '刚果',
'CK': '库克群岛',
'CR': '哥斯达黎加',
'CI': '科特迪瓦',
'HR': '克罗地亚',
'CU': '古巴',
'CW': '库拉索',
'CY': '塞浦路斯',
'CZ': '捷克共和国',
'DK': '丹麦',
'DJ': '吉布提',
'DM': '多米尼克',
'DO': '多米尼加共和国',
'EC': '厄瓜多尔',
'EG': '埃及',
'SV': '萨尔瓦多',
'GQ': '赤道几内亚',
'ER': '厄立特里亚',
'EE': '爱沙尼亚',
'SZ': '斯威士兰',
'ET': '埃塞俄比亚',
'FK': '福克兰群岛（马尔维纳斯）',
'FO': '法罗群岛',
'FJ': '斐济',
'FI': '芬兰',
'FR': '法国',
'GF': '法属圭亚那',
'PF': '法属波利尼西亚',
'TF': '法属南部领地',
'GA': '加蓬',
'GM': '冈比亚',
'GE': '格鲁吉亚',
'DE': '德国',
'GH': '加纳',
'GI': '直布罗陀',
'GR': '希腊',
'GL': '格陵兰',
'GD': '格林纳达',
'GP': '瓜德罗普',
'GU': '关岛',
'GT': '危地马拉',
'GG': '根西',
'GN': '几内亚',
'GW': '几内亚比绍',
'GY': '圭亚那',
'HT': '海地',
'HM': '赫德岛和麦克唐纳群岛',
'VA': '梵蒂冈',
'HN': '洪都拉斯',
'HK': '香港',

'HU': '匈牙利',
'IS': '冰岛',
'IN': '印度',
'ID': '印度尼西亚',
'IR': '伊朗',
'IQ': '伊拉克',
'IE': '爱尔兰',
'IM': '马恩岛',
'IL': '以色列',
'IT': '意大利',
'JM': '牙买加',
'JP': '日本',
'JE': '泽西',
'JO': '约旦',
'KZ': '哈萨克斯坦',
'KE': '肯尼亚',
'KI': '基里巴斯',
'KP': '朝鲜',
'KR': '韩国',
'KW': '科威特',
'KG': '吉尔吉斯斯坦',
'LA': '老挝人民民主共和国',
'LV': '拉脱维亚',
'LB': '黎巴嫩',
'LS': '莱索托',
'LR': '利比里亚',
'LY': '利比亚',
'LI': '列支敦士登',
'LT': '立陶宛',
'LU': '卢森堡',
'MO': '澳门',
'MG': '马达加斯加',
'MW': '马拉维',
'MY': '马来西亚',
'MV': '马尔代夫',
'ML': '马里',
'MT': '马耳他',
'MH': '马绍尔群岛',
'MQ': '马提尼克',
'MR': '毛里塔尼亚',
'MU': '毛里求斯',
'YT': '马约特',
'MX': '墨西哥',
'FM': '密克罗尼西亚联邦',
'MD': '摩尔多瓦',
'MC': '摩纳哥',
'MN': '蒙古',
'ME': '黑山',
'MS': '蒙特塞拉特',
'MA': '摩洛哥',
'MZ': '莫桑比克',
'MM': '缅甸',
'NA': '纳米比亚',
'NR': '瑙鲁',
'NP': '尼泊尔',
'NL': '荷兰',
'NC': '新喀里多尼亚',
'NZ': '新西兰',
'NI': '尼加拉瓜',
'NE': '尼日尔',
'NG': '尼日利亚',
'NU': '纽埃',
'NF': '诺福克岛',
'MK': '北马其顿',
'MP': '北马里亚纳群岛',
'NO': '挪威',

'OM': '阿曼',
'PK': '巴基斯坦',
'PW': '帕劳',
'PS': '巴勒斯坦',
'PA': '巴拿马',
'PG': '巴布亚新几内亚',
'PY': '巴拉圭',
'PE': '秘鲁',
'PH': '菲律宾',
'PN': '皮特凯恩',
'PL': '波兰',
'PT': '葡萄牙',
'PR': '波多黎各',
'QA': '卡塔尔',
'RE': '留尼汪',
'RO': '罗马尼亚',
'RU': '俄罗斯',
'RW': '卢旺达',
'BL': '圣巴泰勒米',
'SH': '圣赫勒拿、阿森松和特里斯坦-达库尼亚',
'KN': '圣基茨和尼维斯',
'LC': '圣卢西亚',
'MF': '法属圣马丁',
'PM': '圣皮埃尔和密克隆',
'VC': '圣文森特和格林纳丁斯',
'WS': '萨摩亚',
'SM': '圣马力诺',
'ST': '圣多美和普林西比',
'SA': '沙特阿拉伯',
'SN': '塞内加尔',
'RS': '塞尔维亚',
'SC': '塞舌尔',
'SL': '塞拉利昂',
'SG': '新加坡',
'SX': '荷属圣马丁',
'SK': '斯洛伐克',
'SI': '斯洛文尼亚',
'SB': '所罗门群岛',
'SO': '索马里',
'ZA': '南非',
'GS': '南乔治亚和南桑威奇群岛',
'SS': '南苏丹',
'ES': '西班牙',
'LK': '斯里兰卡',
'SD': '苏丹',
'SR': '苏里南',
'SJ': '斯瓦尔巴和扬马延',
'SE': '瑞典',
'CH': '瑞士',
'SY': '叙利亚',
'TW': '台湾',
'TJ': '塔吉克斯坦',
'TZ': '坦桑尼亚',
'TH': '泰国',
'TL': '东帝汶',
'TG': '多哥',
'TK': '托克劳',
'TO': '汤加',
'TT': '特立尼达和多巴哥',
'TN': '突尼斯',
'TR': '土耳其',
'TM': '土库曼斯坦',
'TC': '特克斯和凯科斯群岛',
'TV': '图瓦卢',
'UG': '乌干达',
'UA': '乌克兰',

```

'AE': '阿拉伯联合酋长国',
'GB': '英国',
'UM': '美国本土外小岛屿',
'US': '美国',
'UY': '乌拉圭',
'UZ': '乌兹别克斯坦',
'VU': '瓦努阿图',
'VE': '委内瑞拉',
'VN': '越南',
'VG': '英属维尔京群岛',
'VI': '美属维尔京群岛',
'WF': '瓦利斯和富图纳',
'EH': '西撒哈拉',
'YE': '也门',
'ZM': '赞比亚',
'ZW': '津巴布韦',
};

// =====
// 文件: lib/core/constants/dharma_assets.dart
// =====

/// 法宝素材列表
///
/// 此文件根据原生Web应用中的 `dharma-assets.js` 文件移植而来。
/// 定义了所有可用于全球法布施的素材。
const List<Map<String, String>> DHARMA_ASSETS = [
  // 内置经文和咒语
  {"name": "《一切如来心秘密全身舍利宝篋印陀罗尼经》", "type": "built_in", "path": "assets/built_in/texts..."},
  {"name": "《僧伽吒经》", "type": "built_in", "path": "assets/built_in/texts/僧伽吒经.txt"},
  {"name": "《金刚般若波罗蜜经》", "type": "built_in", "path": "assets/built_in/texts/金刚经.txt"},
  {"name": "《佛说阿弥陀经》", "type": "built_in", "path": "assets/built_in/texts/佛说阿弥陀经.txt"},
  {"name": "《观世音菩萨普门品》", "type": "built_in", "path": "assets/built_in/texts/普门品.txt"},
  {"name": "《高王观世音经》", "type": "built_in", "path": "assets/built_in/texts/高王观世音经.txt"},
  {"name": "《心经》", "type": "built_in", "path": "assets/built_in/texts/心经.txt"},
  {"name": "《大悲咒》", "type": "built_in", "path": "assets/built_in/mantras/大悲咒.txt"},
  {"name": "《楞严咒》", "type": "built_in", "path": "assets/built_in/mantras/楞严咒.txt"},
  {"name": "《宝篋印陀罗尼》", "type": "built_in", "path": "assets/built_in/mantras/宝篋印陀罗尼.tx..."},
  {"name": "《僧伽吒经核心四句偈》", "type": "built_in", "path": "assets/built_in/mantras/僧伽吒经四句..."},
  {"name": "《六字大明咒》", "type": "built_in", "path": "assets/built_in/mantras/六字大明咒.txt"},
  // 存放在Cloudflare R2的大文件
  {"name": "《乾隆大藏经》完整TXT版 (1.1GB)", "type": "r2-asset", "path": "乾隆大藏经/乾隆大藏经txt版.zip"},
  {"name": "《房山石经》陀罗尼梵音 (2.6GB)", "type": "r2-asset", "path": "乾隆大藏经/房山石经陀罗尼梵音音频.zip"},
];

// =====
// 文件: lib/core/constants/route_constants.dart
// =====

class RouteConstants {
  RouteConstants._();

  static const String home = '/';
  static const String login = '/login';
  static const String register = '/register';
  static const String forgotPassword = '/forgot-password';
  static const String profile = '/profile';
  static const String membership = '/membership';
  static const String settings = '/settings';
  static const String leaderboard = '/leaderboard';
  static const String videoFeed = '/video-feed';
  static const String dharmaContent = '/dharma';
  static const String transfer = '/transfer';
}

```

```
// =====  
// 文件: lib/core/design_system/app_theme.dart  
// =====  
  
import 'package:flutter/material.dart';  
import 'colors.dart';  
import 'dart:ui';  
  
class AppTheme {  
  // Primary Colors  
  static const Color primaryColor = nebulaPurple;  
  static const Color secondaryColor = spaceBlue;  
  static const Color accentColor = cosmicGold;  
  static const Color alipayBlue = Color(0xFF1677FF);  
  
  // Gradients  
  static const LinearGradient cosmicGradient = LinearGradient(  
    begin: Alignment.topLeft,  
    end: Alignment.bottomRight,  
    colors: [spaceDeepBlue, Color(0xFF1A237E)], // Deep Blue to Deep Indigo  
  );  
  
  static const LinearGradient nebulaGradient = LinearGradient(  
    begin: Alignment.topLeft,  
    end: Alignment.bottomRight,  
    colors: [nebulaPurple, nebulaPink],  
  );  
  
  static const LinearGradient primaryGradient = cosmicGradient;  
  
  // Glassmorphism Decorations  
  static BoxDecoration glassDecoration = BoxDecoration(  
    color: glassSurface,  
    borderRadius: BorderRadius.circular(20),  
    border: Border.all(color: glassBorder, width: 1),  
    boxShadow: [  
      BoxShadow(  
        color: Colors.black.withOpacity(0.2),  
        blurRadius: 20,  
        spreadRadius: 5,  
      ),  
    ],  
  );  
  
  // Helper to get a glass effect (requires ClipRRect / BackdropFilter usage in widge...  
  // but here we define the container style.  
  
  // Dark Theme (The Main Cosmic Theme)  
  static ThemeData darkTheme = ThemeData(  
    useMaterial3: true,  
    brightness: Brightness.dark,  
    scaffoldBackgroundColor: Colors.transparent, // Important for background image/gr...  
    fontFamily: 'NotoSansSC',  
  
    colorScheme: const ColorScheme.dark(  
      primary: primaryColor,  
      secondary: secondaryColor,  
      tertiary: accentColor,  
      surface: Color(0xFF1E1E2C), // Fallback surface  
      background: Colors.transparent,  
    ),  
  
    appBarTheme: const AppBarTheme(  
      centerTitle: true,
```

```

    elevation: 0,
    backgroundColor: Colors.transparent, // Glass effect usually
    foregroundColor: starlightWhite,
    titleTextStyle: TextStyle(
      fontSize: 20,
      fontWeight: FontWeight.bold,
      color: starlightWhite,
      fontFamily: 'NotoSansSC',
    ),
  ),

  cardTheme: CardThemeData(
    elevation: 0, // We use glass decoration instead usually
    color: glassSurface,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(20),
      side: const BorderSide(color: glassBorder, width: 0.5),
    ),
    margin: const EdgeInsets.symmetric(vertical: 8, horizontal: 4),
  ),

  elevatedButtonTheme: ElevatedButtonThemeData(
    style: ElevatedButton.styleFrom(
      elevation: 10,
      shadowColor: nebulaPurple.withOpacity(0.5),
      backgroundColor: primaryColor, // Use gradient in widget if possible, but her...
      foregroundColor: Colors.white,
      padding: const EdgeInsets.symmetric(horizontal: 32, vertical: 16),
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(30)),
      textStyle: const TextStyle(
        fontSize: 16,
        fontWeight: FontWeight.bold,
        fontFamily: 'NotoSansSC',
      ),
    ),
  ),

  inputDecorationTheme: InputDecorationTheme(
    filled: true,
    fillColor: glassSurface,
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(16),
      borderSide: const BorderSide(color: glassBorder),
    ),
    enabledBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(16),
      borderSide: const BorderSide(color: glassBorder),
    ),
    focusedBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(16),
      borderSide: const BorderSide(color: accentColor, width: 1),
    ),
    hintStyle: TextStyle(color: starlightWhite.withOpacity(0.5)),
    labelStyle: const TextStyle(color: starlightWhite),
  ),

  // Keep existing text themes but ensure colors are correct
  textTheme: const TextTheme(
    bodyLarge: TextStyle(color: starlightWhite),
    bodyMedium: TextStyle(color: starlightWhite),
    titleLarge: TextStyle(color: starlightWhite, fontWeight: FontWeight.bold),
  ),
);

// Light Theme (Keeping it but making it compatible if needed, or just redirect to ...
// For this request, we might want to force Dark mode or make Light mode also "Spac...
```

```
// Let's stick to the user request "Space travel" -> Dark.
static ThemeData lightTheme = darkTheme; // Force Dark/Space theme for now as reque...
}

// =====
// 文件: lib/core/design_system/colors.dart
// =====

import 'package:flutter/material.dart';

const Color white = Colors.white;
const Color black = Colors.black;
const Color red = Colors.red;

// Cosmic / Space Theme Colors
const Color spaceDeepBlue = Color(0xFF0B0E14); // Deepest space background
const Color spaceBlue = Color(0xFF1B263B); // Slightly lighter space
const Color starlightWhite = Color(0xFFE8EAF6); // Star-like white
const Color nebulaPurple = Color(0xFF7B1FA2); // Nebula purple
const Color nebulaPink = Color(0xFFE91E63); // Nebula pink
const Color cosmicGold = Color(0xFFFFD700); // Golden accents (stars/buddha)
const Color glassBorder = Color(0x26FFFFFF); // 15% white for glass borders
const Color glassSurface = Color(0x1AFFFFFF); // 10% white for glass surface

// =====
// 文件: lib/core/di/injection.dart
// =====

import 'package:get_it/get_it.dart';
import '../network/api_client.dart';
import '../features/auth/data/datasources/auth_remote_datasource.dart';
import '../features/auth/data/repositories/auth_repository_impl.dart';
import '../features/auth/domain/repositories/auth_repository.dart';
import '../features/auth/domain/usecases/login_usecase.dart';

final getIt = GetIt.instance;

/// 依赖注入配置
void setupDependencies() {
  // 核心服务
  getIt.registerLazySingleton<ApiClient>(() => ApiClient());

  // 认证模块
  getIt.registerLazySingleton<AuthRemoteDataSource>(() => AuthRemoteDataSourceImpl(ge...
  getIt.registerLazySingleton<AuthRepository>(() => AuthRepositoryImpl(getIt()));
  getIt.registerLazySingleton(() => LoginUseCase(getIt()));
}

// =====
// 文件: lib/core/errors/exceptions.dart
// =====

/// 自定义异常类
class AppException implements Exception {
  final String message;
  final String? code;

  AppException(this.message, [this.code]);

  @override
  String toString() => message;
}
```

```

class NetworkException extends AppException {
  NetworkException([String? message]) : super(message ?? '网络连接失败');
}

class ServerException extends AppException {
  ServerException([String? message]) : super(message ?? '服务器错误');
}

class AuthException extends AppException {
  AuthException([String? message]) : super(message ?? '认证失败');
}

class ValidationException extends AppException {
  ValidationException([String? message]) : super(message ?? '数据验证失败');
}

class CacheException extends AppException {
  CacheException([String? message]) : super(message ?? '缓存错误');
}

// =====
// 文件: lib/core/errors/failures.dart
// =====

import 'package:equatable/equatable.dart';

/// 失败基类
abstract class Failure extends Equatable {
  final String message;

  const Failure(this.message);

  @override
  List<Object> get props => [message];
}

class NetworkFailure extends Failure {
  const NetworkFailure([String message = '网络连接失败']) : super(message);
}

class ServerFailure extends Failure {
  const ServerFailure([String message = '服务器错误']) : super(message);
}

class AuthFailure extends Failure {
  const AuthFailure([String message = '认证失败']) : super(message);
}

class ValidationFailure extends Failure {
  const ValidationFailure([String message = '数据验证失败']) : super(message);
}

class CacheFailure extends Failure {
  const CacheFailure([String message = '缓存错误']) : super(message);
}

// =====
// 文件: lib/core/locations.dart
// =====

class Location {
  final String name;
  final double lat;
  final double lon;

```

```
const Location(this.name, this.lat, this.lon);
}

const List<Location> globalLocations = [
    // 亚洲
    Location("中国", 39.9042, 116.4074),
    Location("日本", 35.6895, 139.6917),
    Location("韩国", 37.5665, 126.9780),
    Location("印度", 28.6139, 77.2090),
    Location("新加坡", 1.3521, 103.8198),
    Location("马来西亚", 3.1390, 101.6869),
    Location("泰国", 13.7563, 100.5018),
    Location("越南", 21.0278, 105.8342),
    Location("印度尼西亚", -6.2088, 106.8456),
    Location("菲律宾", 14.5995, 120.9842),
    Location("沙特阿拉伯", 24.7136, 46.6753),
    Location("阿联酋", 25.276987, 55.296249),
    Location("土耳其", 39.9334, 32.8600),
    Location("以色列", 31.7683, 35.2137),
    Location("巴基斯坦", 30.3753, 69.3451),

    // 欧洲
    Location("俄罗斯", 55.7558, 37.6173),
    Location("德国", 52.5200, 13.4050),
    Location("英国", 51.5074, -0.1278),
    Location("法国", 48.8566, 2.3522),
    Location("意大利", 41.9028, 12.4964),
    Location("西班牙", 40.4168, -3.7038),
    Location("乌克兰", 50.4501, 30.5234),
    Location("波兰", 52.2297, 21.0122),
    Location("荷兰", 52.3676, 4.9041),
    Location("比利时", 50.8503, 4.3517),
    Location("瑞典", 59.3293, 18.0686),
    Location("瑞士", 46.8182, 8.2275),
    Location("挪威", 59.9139, 10.7522),
    Location("希腊", 37.9838, 23.7275),
    Location("芬兰", 60.1699, 24.9384),

    // 北美洲
    Location("美国", 38.9637, -95.7129),
    Location("加拿大", 56.1304, -106.3468),
    Location("墨西哥", 23.6345, -102.5528),

    // 南美洲
    Location("巴西", -14.2350, -51.9253),
    Location("阿根廷", -38.4161, -63.6167),
    Location("哥伦比亚", 4.7110, -74.0721),
    Location("智利", -35.6751, -71.5430),
    Location("秘鲁", -9.1900, -75.0152),

    // 非洲
    Location("尼日利亚", 9.0820, 8.6753),
    Location("埃及", 26.8206, 30.8025),
    Location("南非", -30.5595, 22.9375),
    Location("肯尼亚", -0.0236, 37.9062),
    Location("埃塞俄比亚", 9.1450, 40.4897),
    Location("摩洛哥", 31.7917, -7.0926),

    // 大洋洲
    Location("澳大利亚", -25.2744, 133.7751),
    Location("新西兰", -40.9006, 174.8860),
];

// 国家代码到地理位置的映射
final Map<String, String> countryLocations = {
```


'AF': '阿富汗',
'AL': '阿尔巴尼亚',
'DZ': '阿尔及利亚',
'AS': '美属萨摩亚',
'AD': '安道尔',
'AO': '安哥拉',
'AI': '安圭拉',
'AQ': '南极洲',
'AG': '安提瓜和巴布达',
'AR': '阿根廷',
'AM': '亚美尼亚',
'AW': '阿鲁巴',
'AU': '澳大利亚',
'AT': '奥地利',
'AZ': '阿塞拜疆',
'BS': '巴哈马',
'BH': '巴林',
'BD': '孟加拉国',
'BB': '巴巴多斯',
'BY': '白俄罗斯',
'BE': '比利时',
'BZ': '伯利兹',
'BJ': '贝宁',
'BM': '百慕大',
'BT': '不丹',
'BO': '玻利维亚',
'BA': '波斯尼亚和黑塞哥维那',
'BW': '博茨瓦纳',
'BV': '布维岛',
'BR': '巴西',
'IO': '英属印度洋领地',
'BN': '文莱',
'BG': '保加利亚',
'BF': '布基纳法索',
'BI': '布隆迪',
'KH': '柬埔寨',
'CM': '喀麦隆',
'CA': '加拿大',
'CV': '佛得角',
'KY': '开曼群岛',
'CF': '中非共和国',
'TD': '乍得',
'CL': '智利',
'CN': '中国',
'CO': '哥伦比亚',
'KM': '科摩罗',
'CG': '刚果（金）',
'CD': '刚果（金）',
'CK': '库克群岛',
'CR': '哥斯达黎加',
'CI': '科特迪瓦',
'HR': '克罗地亚',
'CU': '古巴',
'CY': '塞浦路斯',
'CZ': '捷克',
'DK': '丹麦',
'DJ': '吉布提',
'DM': '多米尼克',
'DO': '多米尼加共和国',
'TL': '东帝汶',
'EC': '厄瓜多尔',
'EG': '埃及',
'SV': '萨尔瓦多',
'GQ': '赤道几内亚',
'ER': '厄立特里亚',
'EE': '爱沙尼亚',

'ET': '埃塞俄比亚',
'FK': '福克兰群岛',
'FO': '法罗群岛',
'FJ': '斐济',
'FI': '芬兰',
'FR': '法国',
'GF': '法属圭亚那',
'PF': '法属波利尼西亚',
'TF': '法属南部领地',
'GA': '加蓬',
'GM': '冈比亚',
'GE': '格鲁吉亚',
'DE': '德国',
'GH': '加纳',
'GI': '直布罗陀',
'GR': '希腊',
'GL': '格陵兰',
'GD': '格林纳达',
'GP': '瓜德罗普',
'GU': '关岛',
'GT': '危地马拉',
'GN': '几内亚',
'GW': '几内亚比绍',
'GY': '圭亚那',
'HT': '海地',
'HM': '赫德岛和麦克唐纳群岛',
'HN': '洪都拉斯',
'HK': '中国香港',
'HU': '匈牙利',
'IS': '冰岛',
'IN': '印度',
'ID': '印度尼西亚',
'IR': '伊朗',
'IQ': '伊拉克',
'IE': '爱尔兰',
'IL': '以色列',
'IT': '意大利',
'JM': '牙买加',
'JP': '日本',
'JO': '约旦',
'KZ': '哈萨克斯坦',
'KE': '肯尼亚',
'KI': '基里巴斯',
'KP': '朝鲜',
'KR': '韩国',
'KW': '科威特',
'KG': '吉尔吉斯斯坦',
'LA': '老挝',
'LV': '拉脱维亚',
'LB': '黎巴嫩',
'LS': '莱索托',
'LR': '利比里亚',
'LY': '利比亚',
'LI': '列支敦士登',
'LT': '立陶宛',
'LU': '卢森堡',
'MO': '中国澳门',
'MK': '北马其顿',
'MG': '马达加斯加',
'MW': '马拉维',
'MY': '马来西亚',
'MV': '马尔代夫',
'ML': '马里',
'MT': '马耳他',
'MH': '马绍尔群岛',
'MQ': '马提尼克',

'MR': '毛里塔尼亚',
'MU': '毛里求斯',
'YT': '马约特',
'MX': '墨西哥',
'FM': '密克罗尼西亚联邦',
'MD': '摩尔多瓦',
'MC': '摩纳哥',
'MN': '蒙古',
'ME': '黑山',
'MS': '蒙特塞拉特',
'MA': '摩洛哥',
'MZ': '莫桑比克',
'MM': '缅甸',
'NA': '纳米比亚',
'NR': '瑙鲁',
'NP': '尼泊尔',
'NL': '荷兰',
'AN': '荷属安的列斯',
'NC': '新喀里多尼亚',
'NZ': '新西兰',
'NI': '尼加拉瓜',
'NE': '尼日尔',
'NG': '尼日利亚',
'NU': '纽埃',
'NF': '诺福克岛',
'MP': '北马里亚纳群岛',
'NO': '挪威',
'OM': '阿曼',
'PK': '巴基斯坦',
'PW': '帕劳',
'PS': '巴勒斯坦领土',
'PA': '巴拿马',
'PG': '巴布亚新几内亚',
'PY': '巴拉圭',
'PE': '秘鲁',
'PH': '菲律宾',
'PN': '皮特凯恩',
'PL': '波兰',
'PT': '葡萄牙',
'PR': '波多黎各',
'QA': '卡塔尔',
'RE': '留尼汪',
'RO': '罗马尼亚',
'RU': '俄罗斯',
'RW': '卢旺达',
'BL': '圣巴泰勒米',
'SH': '圣赫勒拿',
'KN': '圣基茨和尼维斯',
'LC': '圣卢西亚',
'MF': '圣马丁',
'PM': '圣皮埃尔和密克隆',
'VC': '圣文森特和格林纳丁斯',
'WS': '萨摩亚',
'SM': '圣马力诺',
'ST': '圣多美和普林西比',
'SA': '沙特阿拉伯',
'SN': '塞内加尔',
'RS': '塞尔维亚',
'SC': '塞舌尔',
'SL': '塞拉利昂',
'SG': '新加坡',
'SK': '斯洛伐克',
'SI': '斯洛文尼亚',
'SB': '所罗门群岛',
'SO': '索马里',
'ZA': '南非',

```

'GS': '南乔治亚和南桑威奇群岛',
'ES': '西班牙',
'LK': '斯里兰卡',
'SD': '苏丹',
'SR': '苏里南',
'SJ': '斯瓦尔巴和扬马延',
'SZ': '斯威士兰',
'SE': '瑞典',
'CH': '瑞士',
'SY': '叙利亚',
'TW': '中国台湾',
'TJ': '塔吉克斯坦',
'TZ': '坦桑尼亚',
'TH': '泰国',
'TG': '多哥',
'TK': '托克劳',
'TO': '汤加',
'TT': '特立尼达和多巴哥',
'TN': '突尼斯',
'TR': '土耳其',
'TM': '土库曼斯坦',
'TC': '特克斯和凯科斯群岛',
'TV': '图瓦卢',
'UG': '乌干达',
'UA': '乌克兰',
'AE': '阿拉伯联合酋长国',
'GB': '英国',
'US': '美国',
'UM': '美国本土外小岛屿',
'UY': '乌拉圭',
'UZ': '乌兹别克斯坦',
'VU': '瓦努阿图',
'VA': '梵蒂冈城',
'VE': '委内瑞拉',
'VN': '越南',
'VG': '英属维尔京群岛',
'VI': '美属维尔京群岛',
'WF': '瓦利斯和富图纳',
'EH': '西撒哈拉',
'YE': '也门',
'ZM': '赞比亚',
'ZW': '津巴布韦',
};

```

```

// =====
// 文件: lib/core/network/api_client.dart
// =====

```

```

import 'dart:convert';
import 'package:http/http.dart' as http;
import '../config/app_config.dart';
import '../constants/api_constants.dart';
import '../errors/exceptions.dart';

```

```

/// 统一的API客户端

```

```

class ApiClient {
  final http.Client _client;
  String? _token;

```

```

  ApiClient({http.Client? client}) : _client = client ?? http.Client();

```

```

  void setToken(String? token) {
    _token = token;
  }

```

```

Map<String, String> get _headers => {
  ApiConstants.headerContentType: ApiConstants.contentTypeJson,
  ApiConstants.headerAccept: ApiConstants.contentTypeJson,
  if (_token != null) ApiConstants.headerAuthorization: 'Bearer $_token',
};

Future<Map<String, dynamic>> get(String endpoint) async {
  try {
    final url = Uri.parse('${AppConfig.apiUrl}$endpoint');
    final response = await _client.get(url, headers: _headers).timeout(AppConfig.re...
    return _handleResponse(response);
  } catch (e) {
    throw NetworkException(e.toString());
  }
}

Future<Map<String, dynamic>> post(String endpoint, Map<String, dynamic> body) async...
try {
  final url = Uri.parse('${AppConfig.apiUrl}$endpoint');
  final response = await _client
    .post(url, headers: _headers, body: jsonEncode(body))
    .timeout(AppConfig.requestTimeout);
  return _handleResponse(response);
} catch (e) {
  throw NetworkException(e.toString());
}
}

Map<String, dynamic> _handleResponse(http.Response response) {
  if (response.statusCode >= 200 && response.statusCode < 300) {
    return jsonDecode(response.body);
  } else if (response.statusCode == 401) {
    throw AuthException('认证失败');
  } else if (response.statusCode >= 500) {
    throw ServerException('服务器错误');
  } else {
    throw AppException('请求失败: ${response.statusCode}');
  }
}

void dispose() {
  _client.close();
}
}

// =====
// 文件: lib/core/startup/deferred_loader.dart
// =====

import 'dart:async';
import 'package:flutter/foundation.dart';

/// 延迟加载器 - 避免启动时的资源竞争
class DeferredLoader {
  static final DeferredLoader _instance = DeferredLoader._internal();
  factory DeferredLoader() => _instance;
  DeferredLoader._internal();

  final Map<String, Timer> _timers = {};
  final Map<String, bool> _loadingStates = {};

  /// 延迟执行任务，避免启动时的资源竞争
  void scheduleTask(String taskId, Duration delay, Future<void> Function() task) {
    // 取消之前的任务
    _timers[taskId]?.cancel();
  }
}

```

```
_timers[taskId] = Timer(delay, () async {
  if (_loadingStates[taskId] == true) return; // 防止重复执行

  _loadingStates[taskId] = true;
  try {
    await task();
    debugPrint(' 延迟任务完成: $taskId');
  } catch (e) {
    debugPrint(' 延迟任务失败: $taskId - $e');
  } finally {
    _loadingStates[taskId] = false;
    _timers.remove(taskId);
  }
});
}

/// 取消指定任务
void cancelTask(String taskId) {
  _timers[taskId]?.cancel();
  _timers.remove(taskId);
  _loadingStates.remove(taskId);
}

/// 清理所有任务
void dispose() {
  for (final timer in _timers.values) {
    timer.cancel();
  }
  _timers.clear();
  _loadingStates.clear();
}
}

// =====
// 文件: lib/core/startup/startup_optimizer.dart
// =====

import 'dart:async';
import 'package:flutter/foundation.dart';

/// 启动优化器 - 避免进程发送冲突造成堵塞
class StartupOptimizer {
  static final StartupOptimizer _instance = StartupOptimizer._internal();
  factory StartupOptimizer() => _instance;
  StartupOptimizer._internal();

  final List<Future<void> Function()> _initQueue = [];
  bool _isInitializing = false;
  final Completer<void> _initCompleter = Completer<void>();

  /// 添加初始化任务到队列
  void addInitTask(Future<void> Function() task) {
    _initQueue.add(task);
  }
}

/// 开始分批初始化，避免阻塞
Future<void> startInitialization() async {
  if (_isInitializing) return _initCompleter.future;

  _isInitializing = true;
  debugPrint(' 启动优化器开始分批初始化');

  try {
    // 分批处理，每批最多3个任务
    const batchSize = 3;
```

```
// 手柄
Container(
  width: 40,
  height: 4,
  margin: const EdgeInsets.symmetric(vertical: 12),
  decoration: BoxDecoration(
    color: Colors.white24,
    borderRadius: BorderRadius.circular(2),
  ),
),

// 标题和关闭按钮
Padding(
  padding: const EdgeInsets.symmetric(horizontal: 16),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween,
    children: [
      const SizedBox(width: 40), // 占位
      const Text(
        '选择修行功课',
        style: TextStyle(
          color: Colors.white,
          fontSize: 22,
          fontWeight: FontWeight.bold,
        ),
      ),
      IconButton(
        icon: const Icon(Icons.close, color: Colors.white54),
        onPressed: () => Navigator.pop(context),
      ),
    ],
  ),
),

// 重要警告提醒
Container(
  margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
  padding: const EdgeInsets.all(12),
  decoration: BoxDecoration(
    gradient: LinearGradient(
      colors: [
        Colors.red.withOpacity(0.2),
        Colors.orange.withOpacity(0.15),
      ],
    ),
  ),
  borderRadius: BorderRadius.circular(12),
  border: Border.all(color: Colors.orange.withOpacity(0.5), width: 1.5),
),
child: const Row(
  children: [
    Icon(Icons.warning_amber_rounded, color: Colors.orange, size: 28),
    SizedBox(width: 12),
    Expanded(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            '重要提醒',
            style: TextStyle(
              color: Colors.orange,
              fontSize: 15,
              fontWeight: FontWeight.bold,
            ),
          ),
          SizedBox(height: 2),
          Text(
```

```

        '一旦确认选择，功课将永久锁定，无法修改！',
        style: TextStyle(
          color: Colors.white70,
          fontSize: 13,
        ),
      ),
    ],
  ],
),
),
],
),
),
),

const Text(
  '请选择一门深入修行的功课',
  textAlign: TextAlign.center,
  style: TextStyle(
    color: Colors.white54,
    fontSize: 14,
  ),
),

// 搜索栏
Padding(
  padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
  child: TextField(
    controller: _searchController,
    style: const TextStyle(color: Colors.white),
    decoration: InputDecoration(
      hintText: '搜索经文标题...',
      hintStyle: const TextStyle(color: Colors.white38),
      prefixIcon: const Icon(Icons.search, color: Colors.white54),
      suffixIcon: _searchQuery.isNotEmpty
        ? IconButton(
            icon: const Icon(Icons.clear, color: Colors.white54),
            onPressed: () {
              _searchController.clear();
              setState(() {
                _searchQuery = '';
                _searchResults.clear();
              });
            },
          )
        : null,
      filled: true,
      fillColor: const Color(0xFF2A2A2A),
      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(12),
        borderSide: BorderSide.none,
      ),
      contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: ...
    ),
  ),
),

const Divider(color: Colors.white12, height: 1),

// 功课列表
Expanded(
  child: _isLoading
    ? const Center(
        child: CircularProgressIndicator(color: Color(0xFFD4AF37)),
      )
    : _searchQuery.isNotEmpty
      ? _buildSearchResults()
      : _buildGroupedList(),

```



```

),
// 底部确认按钮
Container(
  padding: const EdgeInsets.all(20),
  decoration: const BoxDecoration(
    color: Color(0xFF1E1E1E),
    border: Border(top: BorderSide(color: Colors.white12)),
  ),
  child: SafeArea(
    top: false,
    child: SizedBox(
      width: double.infinity,
      height: 56,
      child: ElevatedButton(
        onPressed: _selectedTitle != null ? _confirmSelection : null,
        style: ElevatedButton.styleFrom(
          backgroundColor: const Color(0xFFD4AF37),
          disabledBackgroundColor: Colors.white12,
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(28),
          ),
        ),
        child: Text(
          _selectedTitle != null ? '确认选择「$_selectedTitle」' : '请先选择一门功课',
          style: TextStyle(
            color: _selectedTitle != null ? Colors.black : Colors.white38,
            fontSize: 16,
            fontWeight: FontWeight.bold,
          ),
        ),
      ),
    ),
  ),
),
);
}

```

```
Widget_buildSearchResults() {
  if (_searchResults.isEmpty) {
    return Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          const Icon(Icons.search_off, size: 48, color: Colors.white24),
          const SizedBox(height: 16),
          Text(
            '未找到 "$_searchQuery" 相关的经文',
            style: const TextStyle(color: Colors.white38, fontSize: 14),
          ),
        ],
      ),
    );
  }
}
```

```
return ListView.builder(
  padding: const EdgeInsets.symmetric(vertical: 8),
  itemCount: _searchResults.length,
  itemBuilder: (context, index) {
    final practice = _searchResults[index];
    return _buildPracticeItem(practice);
  },
);
}
```

```

Widget _buildGroupedList() {
  if (_groupedPractices.isEmpty) {
    return const Center(
      child: Text('暂无经文素材', style: TextStyle(color: Colors.white38)),
    );
  }

  return ListView.builder(
    padding: const EdgeInsets.symmetric(vertical: 8),
    itemCount: _groupedPractices.length,
    itemBuilder: (context, index) {
      final entry = _groupedPractices.entries.elementAt(index);
      final directory = entry.key;
      final practices = entry.value;
      final isExpanded = _expandedGroups.contains(directory);
      final dirName = directory.split('/').last;

      return Column(
        children: [
          // 分组标题
          ListTile(
            contentPadding: const EdgeInsets.symmetric(horizontal: 20),
            leading: Container(
              width: 40,
              height: 40,
              decoration: BoxDecoration(
                color: const Color(0xFF2A2A2A),
                borderRadius: BorderRadius.circular(8),
              ),
            ),
            child: Icon(
              isExpanded ? Icons.folder_open : Icons.folder,
              color: const Color(0xFFD4AF37),
              size: 20,
            ),
          ),
          title: Text(
            dirName,
            style: const TextStyle(color: Colors.white, fontWeight: FontWeight.bo...
          ),
          subtitle: Text(
            '${practices.length} 个经文',
            style: const TextStyle(color: Colors.white38, fontSize: 12),
          ),
          trailing: Icon(
            isExpanded ? Icons.expand_less : Icons.expand_more,
            color: Colors.white54,
          ),
          onTap: () {
            setState(() {
              if (isExpanded) {
                _expandedGroups.remove(directory);
              } else {
                _expandedGroups.add(directory);
              }
            });
          },
        ],
        // 展开的内容
        if (isExpanded)
          ...practices.map((p) => _buildPracticeItem(p)),
      );
    },
  );
}

```

```

Widget _buildPracticeItem(Map<String, dynamic> practice) {
  final title = practice['name'] ?? '未知';
  final filePath = practice['key'] ?? '';
  final isSelected = _selectedTitle == title && _selectedFilePath == filePath;

  return ListTile(
    contentPadding: const EdgeInsets.symmetric(horizontal: 20, vertical: 4),
    leading: Container(
      width: 44,
      height: 44,
      decoration: BoxDecoration(
        color: isSelected
          ? const Color(0xFFD4AF37).withOpacity(0.2)
          : const Color(0xFF2A2A2A),
        borderRadius: BorderRadius.circular(10),
        border: isSelected
          ? Border.all(color: const Color(0xFFD4AF37), width: 2)
          : null,
      ),
      child: Icon(
        Icons.auto_stories,
        color: isSelected ? const Color(0xFFD4AF37) : Colors.white54,
        size: 22,
      ),
    ),
    title: Text(
      title,
      style: TextStyle(
        color: isSelected ? const Color(0xFFD4AF37) : Colors.white,
        fontSize: 16,
        fontWeight: isSelected ? FontWeight.bold : FontWeight.normal,
      ),
      maxLines: 2,
      overflow: TextOverflow.ellipsis,
    ),
    trailing: isSelected
      ? const Icon(Icons.check_circle, color: Color(0xFFD4AF37), size: 26)
      : const Icon(Icons.circle_outlined, color: Colors.white24, size: 26),
    onTap: () {
      setState(() {
        _selectedTitle = title;
        _selectedFilePath = filePath;
      });
    },
  );
}

/// 显示功课选择弹窗（可关闭）
Future<void> showPracticeSelectionSheet(BuildContext context, {VoidCallback? onSelect...}) {
  return showModalBottomSheet(
    context: context,
    isScrollControlled: true,
    isDismissible: true,
    enableDrag: true,
    backgroundColor: Colors.transparent,
    builder: (context) => PracticeSelectionSheet(onSelected: onSelect),
  );
}

// =====
// 文件: lib/widgets/practice_stats_card.dart
// =====

```

```
import 'package:flutter/material.dart';
import '../services/practice_stats_service.dart';

/// 修行统计卡片 - 一念计数风格
class PracticeStatsCard extends StatefulWidget {
  final VoidCallback? onTapRecord; // 补录功课
  final VoidCallback? onTapHistory; // 查看记录
  final VoidCallback? onTapDedication; // 发愿回向
  final VoidCallback? onTapSettings; // 功课设置

  const PracticeStatsCard({
    super.key,
    this.onTapRecord,
    this.onTapHistory,
    this.onTapDedication,
    this.onTapSettings,
  });

  @override
  State<PracticeStatsCard> createState() => _PracticeStatsCardState();
}

class _PracticeStatsCardState extends State<PracticeStatsCard> {
  bool _showWeekly = true; // true=周, false=月
  final _service = PracticeStatsService();

  @override
  void initState() {
    super.initState();
    _loadData();
  }

  Future<void> _loadData() async {
    await _service.loadAllData();
    if (mounted) setState(() {});
  }

  @override
  Widget build(BuildContext context) {
    return ListenableBuilder(
      listenable: _service,
      builder: (context, _) {
        final stats = _service.stats;
        final goals = _service.goals;
        final weeklyData = _service.weeklyData;

        return Container(
          decoration: BoxDecoration(
            color: const Color(0xFF1E1E1E),
            borderRadius: BorderRadius.circular(16),
          ),
          child: Column(
            children: [
              // 顶部统计卡片 - 红色渐变背景
              _buildHeaderCard(stats),

              // 功能按钮
              _buildActionButtons(),

              // 发愿目标进度
              if (goals.isNotEmpty) _buildGoalProgress(goals.first),

              // 周/月统计
              _buildPeriodStats(weeklyData),

              // 连续打卡
            ],
          ),
        );
      },
    );
  }
}
```

```

        _buildStreakInfo(stats.consecutiveDays),
      ],
    ),
  );
},
);
}

/// 顶部红色统计卡片
Widget _buildHeaderCard(PracticeStats stats) {
  return Container(
    width: double.infinity,
    padding: const EdgeInsets.all(20),
    decoration: const BoxDecoration(
      gradient: LinearGradient(
        colors: [Color(0xFFE74C3C), Color(0xFFC0392B)],
        begin: Alignment.topLeft,
        end: Alignment.bottomRight,
      ),
      borderRadius: BorderRadius.only(
        topLeft: Radius.circular(16),
        topRight: Radius.circular(16),
      ),
    ),
    child: Column(
      children: [
        // 编辑图标和功课名
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            const Icon(Icons.edit_outlined, color: Colors.white54, size: 18),
            Text(
              stats.today.sutra ?? '选择功课',
              style: const TextStyle(color: Colors.white, fontSize: 16, fontWeight: ...),
            ),
            const SizedBox(width: 18),
          ],
        ),
        const SizedBox(height: 12),

        // 今日修行数
        Text(
          '${stats.today.count}',
          style: const TextStyle(
            color: Colors.white,
            fontSize: 56,
            fontWeight: FontWeight.bold,
            letterSpacing: -2,
          ),
        ),
        const Text('今日', style: TextStyle(color: Colors.white70, fontSize: 14)),

        const SizedBox(height: 20),

        // 累计统计
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceEvenly,
          children: [
            _buildStatColumn('${stats.total.count}', '累计(部)'),
            _buildStatColumn('${stats.total.days}', '累计天数'),
            _buildStatColumn('${stats.consecutiveDays}', '连续天数'),
          ],
        ),

        const SizedBox(height: 16),
      ],
    ),
  );
}

```

```

// 查看报告和榜单按钮
Row(
  children: [
    Expanded(
      child: TextButton(
        onPressed: widget.onTapHistory,
        style: TextButton.styleFrom(
          foregroundColor: Colors.white,
          backgroundColor: Colors.white.withOpacity(0.1),
          padding: const EdgeInsets.symmetric(vertical: 12),
          shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular...
        ),
        child: const Text('查看报告'),
      ),
    ),
    const SizedBox(width: 12),
    Expanded(
      child: TextButton(
        onPressed: () {
          ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('修行榜单功能开发中')),
          );
        },
        style: TextButton.styleFrom(
          foregroundColor: Colors.white,
          backgroundColor: Colors.white.withOpacity(0.1),
          padding: const EdgeInsets.symmetric(vertical: 12),
          shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular...
        ),
        child: const Text('查看榜单'),
      ),
    ),
  ],
),
],
),
],
);
}

```

```

Widget _buildStatColumn(String value, String label) {
  return Column(
    children: [
      Text(
        value,
        style: const TextStyle(
          color: Colors.white,
          fontSize: 24,
          fontWeight: FontWeight.bold,
        ),
      ),
      const SizedBox(height: 4),
      Text(label, style: const TextStyle(color: Colors.white70, fontSize: 12)),
    ],
  );
}

```

```

/// 功能按钮行
Widget _buildActionButtons() {
  return Padding(
    padding: const EdgeInsets.symmetric(vertical: 16),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.spaceEvenly,
      children: [
        _buildActionButton(Icons.add_box_outlined, '补课功课', widget.onTapRecord),
        _buildActionButton(Icons.history, '查改记录', widget.onTapHistory),
        _buildActionButton(Icons.spa_outlined, '发愿回向', widget.onTapDedication),
      ],
    ),
  );
}

```

```

    _buildActionButton(Icons.settings_outlined, '功课设置', widget.onTapSettings),
  ],
),
);
}

```

```

Widget _buildActionButton(IconData icon, String label, VoidCallback? onTap) {
  return GestureDetector(
    onTap: onTap,
    child: Column(
      children: [
        Container(
          width: 48,
          height: 48,
          decoration: BoxDecoration(
            color: const Color(0xFF2A2A2A),
            borderRadius: BorderRadius.circular(12),
          ),
          child: Icon(icon, color: Colors.white70, size: 22),
        ),
        const SizedBox(height: 8),
        Text(label, style: const TextStyle(color: Colors.white70, fontSize: 12)),
      ],
    ),
  );
}

```

/// 发愿目标进度

```

Widget _buildGoalProgress(PracticeGoal goal) {
  return Padding(
    padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            Text(
              '发愿目标: ${goal.currentCount} / ${_formatNumber(goal.targetCount)}',
              style: const TextStyle(color: Colors.white, fontSize: 14),
            ),
            Text(
              '${goal.progress}%',
              style: const TextStyle(color: Colors.white54, fontSize: 14),
            ),
          ],
        ),
        const SizedBox(height: 8),
        ClipRRect(
          borderRadius: BorderRadius.circular(4),
          child: LinearProgressIndicator(
            value: goal.progress / 100,
            minHeight: 6,
            backgroundColor: const Color(0xFF2A2A2A),
            valueColor: const AlwaysStoppedAnimation<Color>(Color(0xFFE74C3C)),
          ),
        ),
      ],
    ),
  );
}

```

/// 周/月统计图表

```

Widget _buildPeriodStats(List<DayStats> weeklyData) {
  return Padding(
    padding: const EdgeInsets.all(16),

```

```

child: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Row(
      mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: [
        Text(
          '本${_showWeekly ? "周" : "月"}: ${_showWeekly ? _service.weekTotal : _s...
          style: const TextStyle(color: Colors.white, fontSize: 14),
        ),
        Container(
          decoration: BoxDecoration(
            color: const Color(0xFF2A2A2A),
            borderRadius: BorderRadius.circular(16),
          ),
          child: Row(
            children: [
              _buildPeriodToggle('周', _showWeekly, () {
                setState(() => _showWeekly = true);
                _service.fetchWeeklyStats();
              }),
              _buildPeriodToggle('月', !_showWeekly, () {
                setState(() => _showWeekly = false);
                _service.fetchMonthlyStats();
              }),
            ],
          ),
        ),
      ],
    ),
    const SizedBox(height: 16),

    // 柱状图
    SizedBox(
      height: 120,
      child: weeklyData.isEmpty
        ? const Center(child: Text('暂无数据', style: TextStyle(color: Colors.whi...
        : Row(
          mainAxisAlignment: MainAxisAlignment.spaceEvenly,
          crossAxisAlignment: CrossAxisAlignment.end,
          children: weeklyData.map((day) => _buildBar(day)).toList(),
        ),
      ),
    ],
  ),
);
}

```

```

Widget _buildPeriodToggle(String label, bool isSelected, VoidCallback onTap) {
  return GestureDetector(
    onTap: onTap,
    child: Container(
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 6),
      decoration: BoxDecoration(
        color: isSelected ? const Color(0xFFE74C3C) : Colors.transparent,
        borderRadius: BorderRadius.circular(16),
      ),
      child: Text(
        label,
        style: TextStyle(
          color: isSelected ? Colors.white : Colors.white54,
          fontSize: 12,
          fontWeight: isSelected ? FontWeight.bold : FontWeight.normal,
        ),
      ),
    ),
  );
}

```



```

    );
}

Widget _buildBar(DayStats day) {
  // 计算柱子高度 (相对于最大值)
  final maxCount = _service.weeklyData.map((d) => d.count).fold(0, (a, b) => a > b ...
  final ratio = maxCount > 0 ? day.count / maxCount : 0.0;
  final height = 80 * ratio + 10; // 最小高度10

  return Column(
    mainAxisAlignment: MainAxisAlignment.end,
    children: [
      Text(
        '${day.count}',
        style: TextStyle(
          color: day.count > 0 ? Colors.white : Colors.white38,
          fontSize: 11,
        ),
      ),
      const SizedBox(height: 4),
      Container(
        width: 28,
        height: height,
        decoration: BoxDecoration(
          gradient: day.count > 0
            ? const LinearGradient(
                colors: [Color(0xFFE74C3C), Color(0xFFC0392B)],
                begin: Alignment.topCenter,
                end: Alignment.bottomCenter,
              )
            : null,
          color: day.count == 0 ? const Color(0xFF2A2A2A) : null,
          borderRadius: BorderRadius.circular(4),
        ),
      ),
      const SizedBox(height: 4),
      Text(
        day.date.substring(5), // MM-DD
        style: const TextStyle(color: Colors.white38, fontSize: 10),
      ),
    ],
  );
}

```

/// 连续打卡信息

```

Widget _buildStreakInfo(int consecutiveDays) {
  return Container(
    padding: const EdgeInsets.all(16),
    decoration: const BoxDecoration(
      border: Border(top: BorderSide(color: Color(0xFF2A2A2A))),
    ),
    child: Row(
      children: [
        const Icon(Icons.local_fire_department, color: Color(0xFFE74C3C), size: 20)...
        const SizedBox(width: 8),
        Text(
          '报数记录: 您已连续打卡 $consecutiveDays 天',
          style: const TextStyle(color: Colors.white, fontSize: 14),
        ),
      ],
    ),
  );
}

```

```

String _formatNumber(int number) {
  if (number >= 10000) {

```

```
    return '${(number / 10000).toStringAsFixed(0)}万';
  }
  return number.toString();
}
}

// =====
// 文件: lib/widgets/recaptcha_dialog.dart
// =====

import 'package:flutter/material.dart';
import 'package:flutter_inappwebview/flutter_inappwebview.dart';
import '../core/config/app_config.dart';

/// Firebase手机验证对话框
/// 使用托管的HTML页面 (Firebase JS SDK) 完成reCAPTCHA验证并发送验证码
class RecaptchaDialog extends StatefulWidget {
  final String phoneNumber;

  const RecaptchaDialog({
    super.key,
    required this.phoneNumber,
  });

  /// 显示对话框，发送验证码后返回sessionInfo
  /// 返回值: {'success': true, 'verificationId': '...'} 或 {'success': false, 'error': '...'}
  static Future<Map<String, dynamic>?> show(BuildContext context, {required String ph...
    return showDialog<Map<String, dynamic>>({
      context: context,
      barrierDismissible: false,
      builder: (context) => RecaptchaDialog(phoneNumber: phoneNumber),
    });
  }

  @override
  State<RecaptchaDialog> createState() => _RecaptchaDialogState();
}

class _RecaptchaDialogState extends State<RecaptchaDialog> {
  bool _isLoading = true;
  String? _error;
  late InAppWebViewController _webViewController;

  // 使用托管的Firebase验证页面URL
  String get _verifyUrl {
    final encodedPhone = Uri.encodeComponent(widget.phoneNumber);
    // 使用Worker托管的验证页面
    return '${AppConfig.apiUrl}/phone-verify.html?phone=$encodedPhone';
  }

  @override
  Widget build(BuildContext context) {
    return Dialog(
      backgroundColor: Colors.transparent,
      child: Container(
        width: 380,
        height: 500,
        decoration: BoxDecoration(
          color: const Color(0xFF1a1a2e),
          borderRadius: BorderRadius.circular(16),
          boxShadow: [
            BoxShadow(
              color: Colors.black.withOpacity(0.5),
              blurRadius: 20,
              offset: const Offset(0, 10),
            ),
          ],
        ),
      ),
    );
  }
}
```

```

    ),
  ],
),
child: ClipRRect(
  borderRadius: BorderRadius.circular(16),
  child: Column(
    children: [
      // 标题栏
      Container(
        padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
        decoration: const BoxDecoration(
          border: Border(
            bottom: BorderSide(color: Colors.white10),
          ),
        ),
        child: Row(
          children: [
            const Icon(Icons.security, color: Colors.white70, size: 20),
            const SizedBox(width: 8),
            const Expanded(
              child: Text(
                '安全验证',
                style: TextStyle(color: Colors.white, fontSize: 16, fontWeigh...
              ),
            ),
            IconButton(
              icon: const Icon(Icons.close, color: Colors.white54, size: 20),
              padding: EdgeInsets.zero,
              constraints: const BoxConstraints(),
              onPressed: () => Navigator.of(context).pop({'success': false, '...
            ),
          ],
        ),
      ),
      // WebView
      Expanded(
        child: Stack(
          children: [
            InAppWebView(
              initialUrlRequest: URLRequest(url: WebUri(_verifyUrl)),
              initialSettings: InAppWebViewSettings(
                javaScriptEnabled: true,
                transparentBackground: true,
                disableContextMenu: true,
                supportZoom: false,
                userAgent: 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) A...
            ),
            onWebViewCreated: (controller) {
              _webViewController = controller;

              // 注册JavaScript处理器接收验证码发送结果
              controller.addJavaScriptHandler(
                handlerName: 'onCodeSent',
                callback: (args) {
                  if (args.isNotEmpty && args[0] is Map) {
                    final result = Map<String, dynamic>.from(args[0]);
                    debugPrint('验证码发送结果: $result');
                    Navigator.of(context).pop(result);
                  }
                  return null;
                },
              );
            },
            onLoadStart: (controller, url) {
              setState(() {
                _isLoading = true;

```

```

        _error = null;
      });
    },
    onLoadStop: (controller, url) {
      setState(() => _isLoading = false);
    },
    onReceivedError: (controller, request, error) {
      setState(() {
        _isLoading = false;
        _error = '加载失败: ${error.description}';
      });
    },
  ),
),
if (_isLoading)
const Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      CircularProgressIndicator(color: Color(0xFFFFE66D)),
      SizedBox(height: 16),
      Text('加载中...', style: TextStyle(color: Colors.white70)),
    ],
  ),
),
if (_error != null)
Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      const Icon(Icons.error_outline, color: Colors.redAccent, ...
      const SizedBox(height: 16),
      Text(_error!, style: const TextStyle(color: Colors.redAcc...
      const SizedBox(height: 16),
      ElevatedButton(
        onPressed: () {
          setState(() => _error = null);
          _webViewController.reload();
        },
        style: ElevatedButton.styleFrom(
          backgroundColor: const Color(0xFFFFE66D),
          foregroundColor: Colors.black,
        ),
        child: const Text('重试'),
      ),
    ],
  ),
),
],
),
),
),
),
),
),
);
}
}

```

```

// =====
// 文件: lib/widgets/reflection_dialog.dart
// =====

```

```

import 'package:flutter/material.dart';
import '../services/comment_service.dart';
import '../services/meditation_session_manager.dart';

```

```

/// 修行心得填写弹窗
///
/// 修行结束后显示，用户可填写心得并发布到对应经文的评论区
class ReflectionDialog extends StatefulWidget {
  final Duration duration;
  final int chantCount;
  final String sutraTitle;
  final String filePath;
  final VoidCallback? onCompleted;

  const ReflectionDialog({
    super.key,
    required this.duration,
    required this.chantCount,
    required this.sutraTitle,
    required this.filePath,
    this.onCompleted,
  });

  @override
  State<ReflectionDialog> createState() => _ReflectionDialogState();
}

class _ReflectionDialogState extends State<ReflectionDialog> {
  final TextEditingController _contentController = TextEditingController();
  final CommentService _commentService = CommentService();
  bool _isSubmitting = false;

  String get _formattedDuration {
    final minutes = widget.duration.inMinutes;
    final seconds = widget.duration.inSeconds % 60;
    return '${minutes.toString().padLeft(2, '0')}:${seconds.toString().padLeft(2, '0')}...
  }

  Future<void> _submitReflection() async {
    final content = _contentController.text.trim();
    if (content.isEmpty) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('请填写修行心得'), backgroundColor: Colors.orange),
      );
      return;
    }

    setState(() => _isSubmitting = true);

    try {
      // 构造评论内容，包含修行时长
      final fullContent = '【修行心得】$content\n\n 修行时长: $_formattedDuration | 念诵: ${widg...

      // 使用 filePath 作为 contentId
      final contentId = 'practice_${widget.filePath.hashCode.abs()}';

      final result = await _commentService.postComment(
        contentId,
        fullContent,
        contentType: widget.sutraTitle,
        filePath: widget.filePath,
        tag: 'practice', // 修行心得标签
      );

      if (mounted) {
        if (result['success'] == true) {
          Navigator.pop(context);
          widget.onCompleted?.call();
          ScaffoldMessenger.of(context).showSnackBar(

```

```

        const SnackBar(
          content: Text('心得已发布，功德无量'),
          backgroundColor: Color(0xFFD4AF37),
        ),
      );
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text(result['error'] ?? '发布失败，请稍后重试'),
          backgroundColor: Colors.red,
        ),
      );
    }
  }
} catch (e) {
  debugPrint('发布心得失败: $e');
  if (mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('网络错误，请稍后重试'), backgroundColor: Colors.red),
    );
  }
} finally {
  if (mounted) {
    setState(() => _isSubmitting = false);
  }
}
}
}

```

```

@override
void dispose() {
  _contentController.dispose();
  super.dispose();
}

```

```

@override
Widget build(BuildContext context) {
  return AlertDialog(
    backgroundColor: const Color(0xFFE1E1E1),
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(20)),
    title: const Row(
      children: [
        Text("", style: TextStyle(fontSize: 28)),
        SizedBox(width: 12),
        Text('功德圆满', style: TextStyle(color: Colors.white, fontSize: 22)),
      ],
    ),
    content: SingleChildScrollView(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // 修行统计
          Container(
            padding: const EdgeInsets.all(16),
            decoration: BoxDecoration(
              gradient: LinearGradient(
                colors: [
                  const Color(0xFFD4AF37).withOpacity(0.2),
                  const Color(0xFFD4AF37).withOpacity(0.05),
                ],
                begin: Alignment.topLeft,
                end: Alignment.bottomRight,
              ),
              borderRadius: BorderRadius.circular(12),
              border: Border.all(color: const Color(0xFFD4AF37).withOpacity(0.3)),
            ),
          ),

```

```

    child: Column(
      children: [
        _buildStatRow(' 功课', widget.sutraTitle),
        const Divider(color: Colors.white12, height: 20),
        _buildStatRow(' 修行时长', _formattedDuration),
        const Divider(color: Colors.white12, height: 20),
        _buildStatRow(' 念诵遍数', '${widget.chantCount}'),
      ],
    ),
  ),
),

const SizedBox(height: 20),

// 心得输入
const Text(
  '记录修行心得（可选）',
  style: TextStyle(color: Colors.white70, fontSize: 14),
),
const SizedBox(height: 8),
TextField(
  controller: _contentController,
  maxLines: 4,
  maxLength: 500,
  style: const TextStyle(color: Colors.white),
  decoration: InputDecoration(
    hintText: '今日修行有何感悟？与同修分享...',
    hintStyle: const TextStyle(color: Colors.white38),
    filled: true,
    fillColor: const Color(0xFF2A2A2A),
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(12),
      borderSide: BorderSide.none,
    ),
    counterStyle: const TextStyle(color: Colors.white38),
  ),
),

const SizedBox(height: 8),
Text(
  '心得将发布到「${widget.sutraTitle}」的评论区',
  style: const TextStyle(color: Colors.white38, fontSize: 12),
),
),
),
actions: [
  TextButton(
    onPressed: _isSubmitting ? null : () {
      Navigator.pop(context);
      widget.onCompleted?.call();
    },
    child: const Text('跳过', style: TextStyle(color: Colors.white54)),
  ),
  ElevatedButton(
    onPressed: _isSubmitting ? null : _submitReflection,
    style: ElevatedButton.styleFrom(
      backgroundColor: const Color(0xFFD4AF37),
      disabledBackgroundColor: Colors.white24,
    ),
    child: _isSubmitting
      ? const SizedBox(
          width: 20,
          height: 20,
          child: CircularProgressIndicator(strokeWidth: 2, color: Colors.whit...
        )
      : const Text('发布心得', style: TextStyle(color: Colors.black)),
  ),

```

```

    ),
  ],
);
}

Widget _buildStatRow(String label, String value) {
  return Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween,
    children: [
      Text(label, style: const TextStyle(color: Colors.white70, fontSize: 14)),
      Text(
        value,
        style: const TextStyle(
          color: Color(0xFFD4AF37),
          fontWeight: FontWeight.bold,
          fontSize: 16,
        ),
      ),
    ],
  ),
];
}
}

```

```

/// 显示修行心得填写弹窗
Future<void> showReflectionDialog(
  BuildContext context, {
    required Duration duration,
    required int chantCount,
    required String sutraTitle,
    required String filePath,
    VoidCallback? onCompleted,
  }) {
  return showDialog(
    context: context,
    barrierDismissible: false,
    builder: (context) => ReflectionDialog(
      duration: duration,
      chantCount: chantCount,
      sutraTitle: sutraTitle,
      filePath: filePath,
      onCompleted: onCompleted,
    ),
  );
}

```

```

// =====
// 文件: lib/widgets/space_background.dart
// =====

```

```

import 'package:flutter/material.dart';
import '../core/design_system/colors.dart';

class SpaceBackground extends StatelessWidget {
  final Widget child;

  const SpaceBackground({super.key, required this.child});

  @override
  Widget build(BuildContext context) {
    return Stack(
      children: [
        // Deep Black Space Background
        Container(
          decoration: const BoxDecoration(
            color: Colors.black,

```



```

        gradient: RadialGradient(
          center: Alignment.center,
          radius: 1.5,
          colors: [
            Color(0xFF0B1026),
            Colors.black,
          ],
          stops: [0.0, 1.0],
        ),
      ),
    ),
  ),

  // Subtle Nebula Effect
  Positioned(
    top: -100,
    right: -100,
    child: Container(
      width: 500,
      height: 500,
      decoration: BoxDecoration(
        shape: BoxShape.circle,
        gradient: RadialGradient(
          colors: [
            nebulaPurple.withOpacity(0.1),
            Colors.transparent,
          ],
        ),
      ),
    ),
  ),
),
),

// Content
child,
],
);
}
}

// =====
// 文件: lib/widgets/sutra_dialog.dart
// =====

import 'package:flutter/material.dart';
import '../models/sutra_model.dart';

class SutraDialog extends StatefulWidget {
  final String title;

  const SutraDialog({Key? key, required this.title}) : super(key: key);

  @override
  State<SutraDialog> createState() => _SutraDialogState();
}

class _SutraDialogState extends State<SutraDialog> with SingleTickerProviderStateMixi...
  late AnimationController _controller;
  late Animation<double> _scaleAnimation;
  late Animation<double> _fadeAnimation;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(vsync: this, duration: const Duration(milliseco...

    _scaleAnimation = CurvedAnimation(parent: _controller, curve: Curves.easeOutBack)...

```

```
_fadeAnimation = CurvedAnimation(parent: _controller, curve: Curves.easeIn);

_controller.forward();
}

@override
void dispose() {
  _controller.dispose();
  super.dispose();
}

String _getSutraContent() {
  final sutra = SutraLibrary.getSutraByTitle(widget.title);
  return sutra?.content ?? '经文内容...';
}

@override
Widget build(BuildContext context) {
  return FadeTransition(
    opacity: _fadeAnimation,
    child: ScaleTransition(
      scale: _scaleAnimation,
      child: Dialog(
        backgroundColor: Colors.transparent,
        child: Container(
          constraints: const BoxConstraints(maxWidth: 600, maxHeight: 700),
          decoration: BoxDecoration(
            gradient: const LinearGradient(
              begin: Alignment.topLeft,
              end: Alignment.bottomRight,
              colors: [Color(0xFFFFF8DC), Color(0xFFFFF0DB), Color(0xFFFFE4B5)],
            ),
            borderRadius: BorderRadius.circular(20),
            border: Border.all(color: const Color(0xFFB8860B), width: 3),
            boxShadow: [
              BoxShadow(color: Colors.black.withOpacity(0.3), blurRadius: 20, spread: 0),
            ],
          ),
          child: Column(
            children: [
              // 标题栏
              Container(
                padding: const EdgeInsets.all(20),
                decoration: BoxDecoration(
                  gradient: LinearGradient(
                    colors: [const Color(0xFFDAA520), const Color(0xFFB8860B)],
                  ),
                  borderRadius: const BorderRadius.only(
                    topLeft: Radius.circular(17),
                    topRight: Radius.circular(17),
                  ),
                ),
              ),
              child: Row(
                children: [
                  const Icon(Icons.auto_stories, color: Colors.white, size: 32),
                  const SizedBox(width: 12),
                  Expanded(
                    child: Text(
                      widget.title,
                      style: const TextStyle(
                        fontSize: 28,
                        fontWeight: FontWeight.bold,
                        color: Colors.white,
                        shadows: [
                          Shadow(color: Colors.black26, blurRadius: 4, offset: Of...
```

```

    ],
  ),
),
IconButton(
  icon: const Icon(Icons.close, color: Colors.white),
  onPressed: () => Navigator.pop(context),
),
],
),
),
),

// 装饰线
Container(
  height: 3,
  decoration: BoxDecoration(
    gradient: LinearGradient(
      colors: [Colors.transparent, const Color(0xFFFFD700), Colors.tr...
    ),
  ),
),

// 经文内容
Expanded(
  child: Container(
    padding: const EdgeInsets.all(24),
    child: SingleChildScrollView(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // 经文分类标签
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 12, verti...
            decoration: BoxDecoration(
              color: const Color(0xFFDAA520).withOpacity(0.2),
              borderRadius: BorderRadius.circular(12),
              border: Border.all(color: const Color(0xFFDAA520), widt...
            ),
            child: Text(
              SutraLibrary.getSutraByTitle(widget.title)?.category ??...
              style: const TextStyle(
                fontSize: 14,
                color: Color(0xFF8B4513),
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
        ],
      ),
      const SizedBox(height: 20),

      // 经文正文
      Text(
        _getSutraContent(),
        style: const TextStyle(
          fontSize: 18,
          height: 2.0,
          color: Color(0xFF3E2723),
          letterSpacing: 1.5,
          fontWeight: FontWeight.w500,
        ),
      ),
    ),
    const SizedBox(height: 30),

    // 底部装饰
    Center(
      child: Container(

```

```
padding: const EdgeInsets.symmetric(horizontal: 20, ver...
decoration: BoxDecoration(
  color: const Color(0xFFFFFD700).withOpacity(0.2),
  borderRadius: BorderRadius.circular(20),
),
child: const Text(
  '南无阿弥陀佛',
  style: TextStyle(
    fontSize: 16,
    color: Color(0xFF8B4513),
    fontWeight: FontWeight.bold,
  ),
),
),
),
),
],
),
),
),
// 底部按钮栏
Container(
  padding: const EdgeInsets.all(16),
  decoration: BoxDecoration(
    color: const Color(0xFFFFF8DC).withOpacity(0.5),
    borderRadius: const BorderRadius.only(
      bottomLeft: Radius.circular(17),
      bottomRight: Radius.circular(17),
    ),
  ),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.spaceEvenly,
    children: [
      _buildActionButton(
        icon: Icons.bookmark_add,
        label: '收藏',
        onPressed: () {
          ScaffoldMessenger.of(
            context,
          ).showSnackBar(const SnackBar(content: Text('已收藏')));
        },
      ),
      _buildActionButton(
        icon: Icons.share,
        label: '分享',
        onPressed: () {
          ScaffoldMessenger.of(
            context,
          ).showSnackBar(const SnackBar(content: Text('分享功能开发中')));
        },
      ),
      _buildActionButton(
        icon: Icons.volume_up,
        label: '朗读',
        onPressed: () {
          ScaffoldMessenger.of(
            context,
          ).showSnackBar(const SnackBar(content: Text('朗读功能开发中')));
        },
      ),
    ],
  ),
),
],
),
```

```

    ),
    ),
    ),
  );
}

Widget _buildActionButton({
  required IconData icon,
  required String label,
  required VoidCallback onPressed,
}) {
  return ElevatedButton.icon(
    onPressed: onPressed,
    icon: Icon(icon, size: 20),
    label: Text(label),
    style: ElevatedButton.styleFrom(
      backgroundColor: const Color(0xFFDAA520),
      foregroundColor: Colors.white,
      padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 12),
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
      elevation: 3,
    ),
  );
}
}

```

```

// =====
// 文件: lib/widgets/trajectoryPainter.dart
// =====

```

```

import 'package:flutter/material.dart';
import 'dart:math' as math;
import 'package:vector_math/vector_math_64.dart' as v;

```

```

class TrajectoryPoint {
  final v.Vector3 position;
  final double timestamp;

  TrajectoryPoint(this.position, this.timestamp);
}

```

```

class TrajectoryPainter extends CustomPainter {
  final double animationValue;
  final v.Vector3 start;
  final v.Vector3 end;
  final v.Vector3 control;
  final Color color;
  final int trailLength;
  final Function(v.Vector3, Size) projectToScreen;

```

```

  TrajectoryPainter({
    required this.animationValue,
    required this.start,
    required this.end,
    required this.control,
    required this.projectToScreen,
    this.color = Colors.cyan,
    this.trailLength = 15,
  });

```

```

  @override
  void paint(Canvas canvas, Size size) {
    if (animationValue <= 0) return;

```

```

    // 计算当前彗星头部位置

```

```

    final currentPos = _getQuadraticBezierPoint(start, control, end, animationValue);
    final screenPos = projectToScreen(currentPos, size);

    // 绘制拖尾
    _drawTrail(canvas, size);

    // 绘制彗星头部
    _drawHead(canvas, screenPos);
}

void _drawTrail(Canvas canvas, Size size) {
  for (int i = 1; i <= trailLength; i++) {
    final t = animationValue - (i * 0.015);
    if (t < 0) break;

    final trailPos = _getQuadraticBezierPoint(start, control, end, t);
    final trailScreenPos = projectToScreen(trailPos, size);

    final opacity = 1.0 - (i / trailLength);
    final radius = 4.0 * opacity;

    final paint = Paint()
      ..color = color.withOpacity(opacity * 0.6)
      ..maskFilter = const MaskFilter.blur(BlurStyle.normal, 3.0);

    canvas.drawCircle(trailScreenPos, radius, paint);
  }
}

void _drawHead(Canvas canvas, Offset screenPos) {
  // 外层辉光
  final glowPaint = Paint()
    ..color = color.withOpacity(0.3)
    ..maskFilter = const MaskFilter.blur(BlurStyle.normal, 8.0);
  canvas.drawCircle(screenPos, 8.0, glowPaint);

  // 中层光晕
  final midPaint = Paint()
    ..color = color.withOpacity(0.7)
    ..maskFilter = const MaskFilter.blur(BlurStyle.normal, 4.0);
  canvas.drawCircle(screenPos, 5.0, midPaint);

  // 核心亮点
  final corePaint = Paint()..color = Colors.white;
  canvas.drawCircle(screenPos, 2.5, corePaint);
}

v.Vector3 _getQuadraticBezierPoint(v.Vector3 p0, v.Vector3 p1, v.Vector3 p2, double...
  final u = 1 - t;
  final tt = t * t;
  final uu = u * u;

  return (p0 * uu) + (p1 * (2 * u * t)) + (p2 * tt);
}

@override
bool shouldRepaint(covariant TrajectoryPainter oldDelegate) {
  return oldDelegate.animationValue != animationValue;
}
}

// =====
// 文件: lib/widgets/transfer_control_bar.dart
// =====

```

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../models/file_transfer_model.dart';

class TransferControlBar extends StatelessWidget {
  final VoidCallback onPauseResume;
  final VoidCallback onCancel;

  TransferControlBar({required this.onPauseResume, required this.onCancel});

  @override
  Widget build(BuildContext context) {
    final model = Provider.of<FileTransferModel>(context);
    final status = model.status;

    return Container(
      height: 60,
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(12),
        boxShadow: [
          BoxShadow(color: Colors.black.withOpacity(0.1), blurRadius: 8, offset: Offs...
        ],
      ),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceEvenly,
        children: [
          // 暂停/继续按钮
          if (status == TransferStatus.transferring || status == TransferStatus.pause...
            _buildControlButton(
              context: context,
              icon: status == TransferStatus.transferring ? Icons.pause : Icons.play_...
              label: status == TransferStatus.transferring ? '暂停' : '继续',
              color: status == TransferStatus.transferring ? Colors.orange : Colors.g...
              onPressed: onPauseResume,
            ),

          // 取消按钮
          if (status != TransferStatus.completed)
            _buildControlButton(
              context: context,
              icon: Icons.cancel,
              label: '取消',
              color: Colors.red,
              onPressed: onCancel,
            ),

          // 完成按钮
          if (status == TransferStatus.completed)
            _buildControlButton(
              context: context,
              icon: Icons.check_circle,
              label: '完成',
              color: Colors.green,
              onPressed: () {
                Navigator.of(context).pop();
              },
            ),
        ],
      ),
    );
  }

  // 构建控制按钮
  Widget _buildControlButton({
    required BuildContext context,
```

```
required IconData icon,
required String label,
required Color color,
required VoidCallback onPressed,
}) {
  return TextButton.icon(
    icon: Icon(icon, color: color, size: 24),
    label: Text(
      label,
      style: TextStyle(color: color, fontWeight: FontWeight.bold),
    ),
    onPressed: onPressed,
    style: TextButton.styleFrom(
      padding: EdgeInsets.symmetric(horizontal: 16, vertical: 8),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(8),
        side: BorderSide(color: color.withOpacity(0.5)),
      ),
      backgroundColor: color.withOpacity(0.1),
    ),
  );
}
```

```
// =====
// 文件: lib/widgets/transfer_control_panel.dart
// =====
```

```
import 'package:flutter/material.dart';
import 'package:file_picker/file_picker.dart';
```

```
/// 传输控制面板
```

```
///
```

```
/// 提供文件选择和传输控制功能的组件。
```

```
class TransferControlPanel extends StatelessWidget {
```

```
  /// 选择文件的回调
```

```
  final VoidCallback onPickFiles;
```

```
  /// 发送文件的回调
```

```
  final VoidCallback onSendFiles;
```

```
  /// 已选择的文件列表
```

```
  final List<PlatformFile> selectedFiles;
```

```
  /// 是否正在发送
```

```
  final bool isSending;
```

```
  /// 是否禁用发送按钮
```

```
  final bool disableSend;
```

```
  /// 构造函数
```

```
  const TransferControlPanel({
```

```
    Key? key,
```

```
    required this.onPickFiles,
```

```
    required this.onSendFiles,
```

```
    required this.selectedFiles,
```

```
    this.isSending = false,
```

```
    this.disableSend = false,
```

```
  }) : super(key: key);
```

```
@override
```

```
Widget build(BuildContext context) {
```

```
  return Card(
```

```
    child: Padding(
```

```
      padding: const EdgeInsets.all(16.0),
```



```

child: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Text('传输控制', style: Theme.of(context).textTheme.titleLarge),
    const SizedBox(height: 16.0),
    Row(
      children: [
        ElevatedButton.icon(
          icon: const Icon(Icons.file_upload),
          label: const Text('选择文件'),
          onPressed: isSending ? null : onPickFiles,
        ),
        const SizedBox(width: 16.0),
        Expanded(
          child: Text(
            selectedFiles.isEmpty ? '未选择文件' : '已选择 ${selectedFiles.length} 个文...',
            style: Theme.of(context).textTheme.bodyMedium,
          ),
        ),
      ],
    ),
  ],
),
if (selectedFiles.isNotEmpty)
  Padding(
    padding: const EdgeInsets.only(top: 8.0),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: selectedFiles
        .map(
          (file) => Padding(
            padding: const EdgeInsets.only(top: 4.0),
            child: Text(
              '${file.name} (${_formatFileSize(file.size)})',
              style: const TextStyle(fontSize: 12.0),
            ),
          ),
        )
        .toList(),
    ),
  ),
const SizedBox(height: 16.0),
ElevatedButton.icon(
  icon: const Icon(Icons.send),
  label: const Text('发送文件'),
  onPressed: (selectedFiles.isEmpty || isSending || disableSend) ? null : ...
  style: ElevatedButton.styleFrom(minimumSize: const Size(double.infinity...
),
if (isSending)
  Padding(
    padding: const EdgeInsets.only(top: 16.0),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: const [
        Text('正在发送...'),
        SizedBox(height: 8.0),
        LinearProgressIndicator(),
      ],
    ),
  ),
],
),
);
}

```

```
/// 格式化文件大小
```

```
String _formatFileSize(int bytes) {
```

```
if (bytes < 1024) {
  return '$bytes B';
} else if (bytes < 1024 * 1024) {
  return '${(bytes / 1024).toStringAsFixed(2)} KB';
} else if (bytes < 1024 * 1024 * 1024) {
  return '${(bytes / (1024 * 1024)).toStringAsFixed(2)} MB';
} else {
  return '${(bytes / (1024 * 1024 * 1024)).toStringAsFixed(2)} GB';
}
}
}

// =====
// 文件: lib/widgets/transfer_mode_selector.dart
// =====

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../models/file_transfer_model.dart';

/// 传输模式选择器
///
/// 允许用户选择不同的传输模式，如全球发送和WiFi广播
class TransferModeSelector extends StatelessWidget {
  const TransferModeSelector({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Consumer<FileTransferModel>(
      builder: (context, model, child) {
        return Card(
          child: Padding(
            padding: const EdgeInsets.all(16.0),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text('法布施设置', style: Theme.of(context).textTheme.titleLarge),
                const SizedBox(height: 16.0),
                _buildLoopingSwitch(
                  context,
                  title: '循环法布施',
                  subtitle: '持续重复传播选定的法布施内容',
                  value: model.isLooping,
                  onChanged: (value) {
                    model.setLooping(value);
                  },
                ),
                const Divider(),
                _buildSpeedControl(context, model),
              ],
            ),
          ),
        );
      },
    );
  }

  Widget _buildModeSwitch(
    BuildContext context, {
    required String title,
    required String subtitle,
    required bool value,
    required ValueChanged<bool> onChanged,
    bool isHighlighted = false,
  }) {
```

```

return ListTile(
  title: Text(
    title,
    style: TextStyle(
      fontWeight: isHighlighted ? FontWeight.bold : FontWeight.normal,
      color: isHighlighted ? Theme.of(context).colorScheme.primary : null,
    ),
  ),
  subtitle: Text(subtitle),
  trailing: Switch(
    value: value,
    onChanged: onChanged,
    activeColor: isHighlighted ? Theme.of(context).colorScheme.primary : null,
  ),
);
}

Widget _buildLoopingSwitch(
  BuildContext context, {
    required String title,
    required String subtitle,
    required bool value,
    required ValueChanged<bool> onChanged,
  }) {
  return ListTile(
    title: Text(title),
    subtitle: Text(subtitle),
    trailing: Switch(value: value, onChanged: onChanged, activeColor: Colors.amber)...
  );
}

Widget _buildSpeedControl(BuildContext context, FileTransferModel model) {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text('发送速度: ${model.sendRateMB.toStringAsFixed(1)} MB/秒'),
      Slider(
        value: model.sendRateMB,
        min: 0.1,
        max: 5.0,
        divisions: 49,
        onChanged: (value) {
          model.setSendRateMB(value);
        },
      ),
    ],
  );
}

// =====
// 文件: lib/widgets/transfer_stats_display.dart
// =====

import 'package:flutter/material.dart';

/// 传输统计显示组件
///
/// 显示文件传输的统计信息，如已发送文件数、已发送数据量等。
class TransferStatsDisplay extends StatelessWidget {
  /// 已发送文件数
  final int sentCount;

  /// 已发送数据量 (MB)
  final double dataSentInMB;

```

```

/// 是否显示标题
final bool showTitle;

/// 构造函数
const TransferStatsDisplay({
  Key? key,
  required this.sentCount,
  required this.dataSentInMB,
  this.showTitle = true,
}) : super(key: key);

@override
Widget build(BuildContext context) {
  return Card(
    child: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          if (showTitle) ...[
            Text('传输统计', style: Theme.of(context).textTheme.titleLarge),
            const SizedBox(height: 16.0),
          ],
          _buildStatRow(context, '已发送节点数:', '$sentCount'),
          const SizedBox(height: 8.0),
          _buildStatRow(context, '已发送数据:', '${dataSentInMB.toStringAsFixed(2)} MB')...
        ],
      ),
    ),
  );
}

/// 构建统计行
Widget _buildStatRow(BuildContext context, String label, String value) {
  return Row(
    children: [
      Text(label, style: const TextStyle(fontWeight: FontWeight.bold)),
      const SizedBox(width: 8.0),
      Text(
        value,
        style: TextStyle(color: Theme.of(context).primaryColor, fontWeight: FontWei...
      ),
    ],
  );
}

// =====
// 文件: lib/widgets/wifi_connection_status.dart
// =====

import 'package:flutter/material.dart';

class WifiConnectionStatus extends StatelessWidget {
  const WifiConnectionStatus({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    // 这个类是空的，作为一个占位符来解决编译错误。
    // 之后会根据需要添加具体实现。
    return const SizedBox.shrink();
  }
}

```